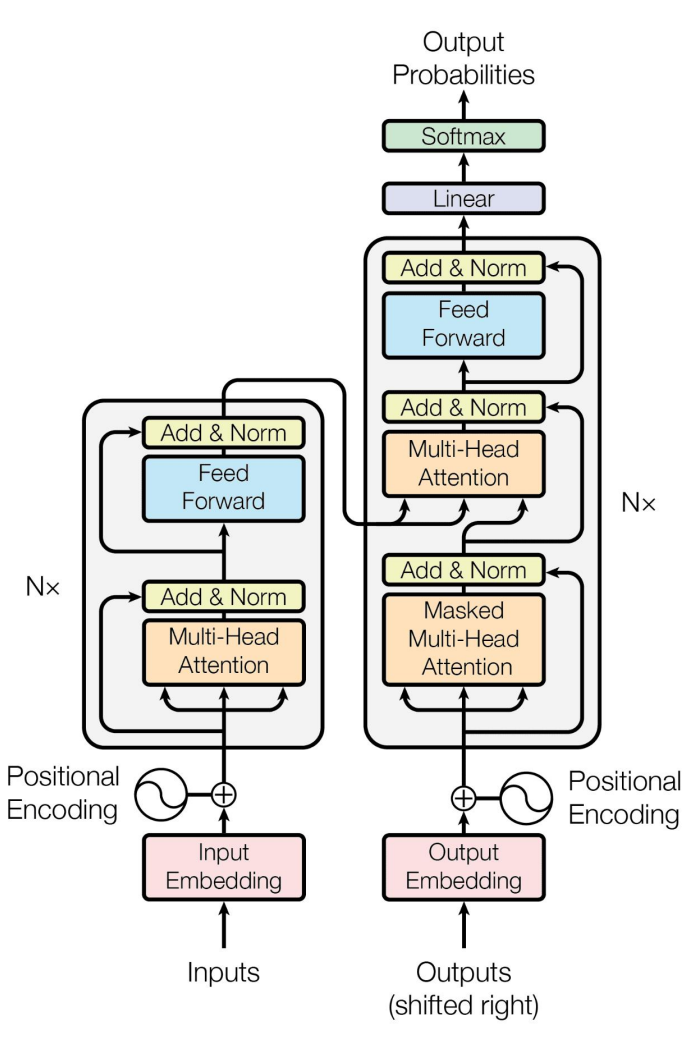
Transformer Model

01

Fundamentals and Applications of Transformer Model

Architecture of Transformer

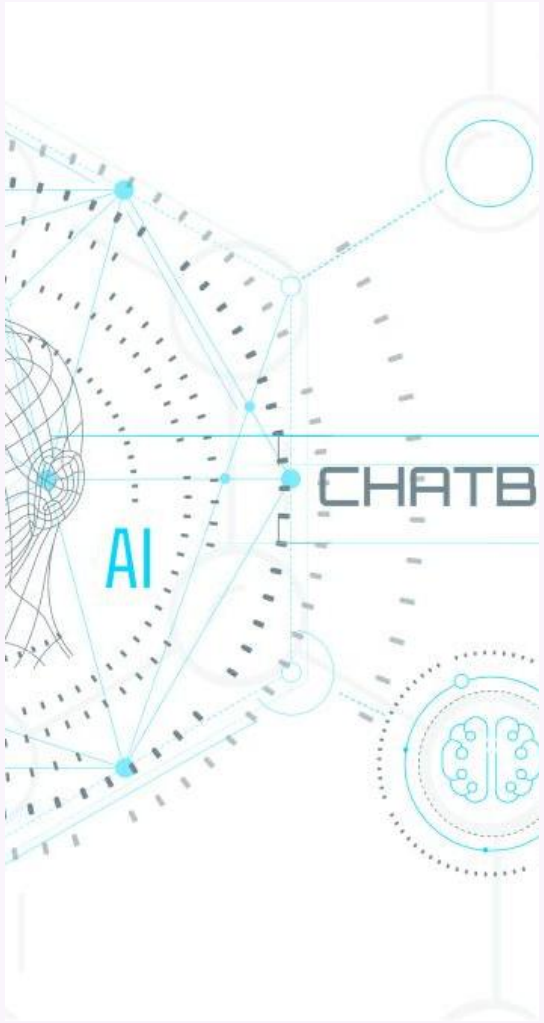


- **Background of Model History:** The rise of Transformer, as a disruptive innovation in the field of NLP, marks a new stage in deep learning technology. Its unique self attention mechanism brings revolutionary improvements to NLP tasks.
- **Overview of Architecture Features:** The Transformer model achieves efficient processing of input sequences through self attention mechanism, and its multi head self attention design further enhances the model's ability to capture complex dependency relationships.
- **Self attention mechanism:** Self attention mechanism is the core of Transformer, which automatically adjusts the relationships between elements by calculating the dot product of query, key, and value vectors, thereby efficiently processing sequence data.
- **Multi head self attention:** The multi head self attention mechanism is an extension of Transformer, which captures multiple dependencies at different positions in the input sequence by processing multiple attention functions in parallel, enhancing the model's representation ability.

Parallel processing sequence capability

- Parallel processing advantage: The parallel processing capability of Transformer comes from its self attention mechanism and multi head self attention design, which enables the model to efficiently process long sequence data during training and inference.
- Sequence processing efficiency: Through parallel computing, Transformer significantly improves the efficiency and accuracy of sequence processing, making it an efficient tool for processing long text data in NLP tasks and promoting the development of NLP technology.
- Model complexity management: Despite having the advantage of parallel processing, the complexity of Transformers increases with sequence length, requiring fine tuning of model size and parameters to balance performance and resource consumption.
- Application scenario expansion: The parallel processing capability of Transformer demonstrates great potential at the application level, not limited to NLP, but gradually expanding to multiple fields such as image recognition and speech recognition, leading a new round of innovation in AI technology.

Breakthrough in NLP tasks



Text generation capability

The introduction of Transformer enables machines to understand and generate text more accurately, bringing unprecedented development opportunities to the NLP field.

Task performance improvement

Significant achievements have been made in various NLP tasks, such as sentiment analysis, question answering systems, machine translation, etc., with performance far exceeding traditional RNNs and CNNs.

New Era of Language Processing

The rise of Transformer marks a new era for NLP, and its influence will continue to drive the field towards smarter and more efficient directions.

The advantages of self attention mechanism

- Parallel processing: Self attention mechanism can achieve parallel processing, thereby improving processing efficiency, while traditional recurrent neural networks (RNNs) are difficult to parallelize, resulting in slower processing speed.
- Long distance dependency processing: The self attention mechanism can more effectively capture long-distance dependency relationships by calculating the attention weights of each position to all other positions.
- Flexibility: The self attention mechanism can flexibly adjust the attention level between different positions by calculating attention weights, thus better adapting to the needs of different tasks.
- Interpretability: The self attention mechanism can clearly indicate the attention level of the model between different positions by calculating attention weights, thereby enhancing the interpretability of the model.

Context comprehension ability

Global context understanding

The Transformer model, through its self attention mechanism, is able to capture global contextual information, thereby more accurately understanding the semantics of the input sequence.

Multi head attention mechanism

By processing multiple attention functions in parallel, it captures multiple dependencies at different positions in the input sequence, enhancing the model's representational power.

Neural network enhancement

Transformer models utilize deep learning techniques to continuously improve their understanding of complex contexts through learning from a large amount of training data.

Widely applicable

Due to its powerful contextual understanding ability, Transformer models have been widely used in fields such as NLP and image recognition, demonstrating excellent performance.

02

Development of Transformer History

Attention Is All You Need

- The birth of Transformer: In 2017, Transformer models emerged, completely revolutionizing the field of NLP. Its biggest highlight is the introduction of self attention mechanism, which can efficiently capture key information in the sequence.
- Self attention mechanism: The self attention mechanism calculates the similarity between each element and all other elements, and weights the sum of the values of these elements to generate a new representation. This mechanism can automatically adapt and process sequences of different lengths.
- Farewell to RNNs: The birth of Transformer models means that when processing sequential data, we no longer rely solely on Recurrent Neural Networks (RNNs), but can adopt a more efficient and powerful model architecture.
- Model advantages: Transformer models effectively capture sequence dependencies through self attention mechanisms, and their training efficiency and model performance are significantly better than traditional RNNs, making them the mainstream model in the NLP field.

Evolution since 2017

- Bidirectional Encoder Representation from Transformers (BERT) : was introduced in 2018, which innovatively adopts a bidirectional encoding mechanism that can simultaneously consider the contextual information of the text.
- Unsupervised Pre training: The GPT (Generative Pre trained Transformer) model was also introduced in 2018. Unlike BERT, GPT adopts an unsupervised pre training approach aimed at capturing language structures through generative tasks.
- Long text processing: Transformer XL (Transformer eXtra Large) was introduced in 2019, solving the problem of Transformer models being difficult to handle long sequences. The introduced loop mechanism and expanded context window.
- Parameter optimization: The ALBERT (A Lite BERT) model was launched in 2019 and utilizes various parameter reduction techniques to significantly reduce memory usage and computational resource requirements while maintaining high performance.

Becoming a Modern AI Infrastructure

Multi domain development

Since its debut in 2017, Transformer models have shone in the NLP field and gradually expanded to multiple fields such as image recognition and speech recognition, demonstrating strong cross domain application capabilities.

In the era of big models

With the improvement of computing power and data accumulation, Transformer models are becoming increasingly large in scale and have become the mainstream choice in the NLP field. Its powerful ability lies in its ability to capture more complex features and model longer dependency relationships.

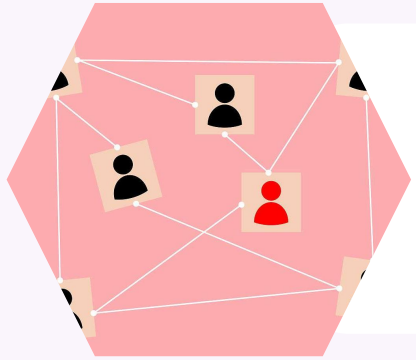
AI infrastructure

Nowadays, Transformer models are not only tools in the NLP field, but also important infrastructure for building modern AI systems. They are capable of handling various complex tasks, from image recognition to voice interaction.

03

Transformer model structure

Encoder module function

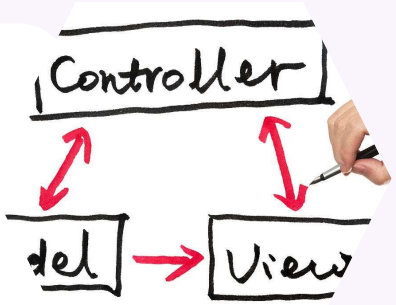


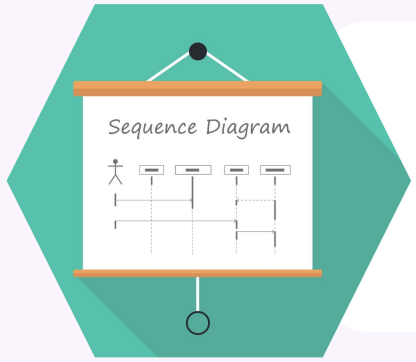
Self attention mechanism

The core is the self attention mechanism, which enables the model to automatically capture key information from the input sequence without explicitly specifying the order of input data.

Multi head attention

By processing multiple self attention functions in parallel, the model's processing efficiency and performance are further improved, enhancing its ability to capture input sequences.





Feedforward network

mainly composed of feedforward neural networks, it is responsible for further extracting feature information from the input sequence and passing this information to the decoder module.

Decoder module function

Decoder core

self attention mechanism, first processes the entire input sequence, and then outputs the target elements one by one, which can automatically adjust the focus on different parts of the input.

Encoder decoder attention

Connect the encoder and decoder to enable the decoder to obtain the hidden state of the encoder, enhancing the model's ability to handle long-distance dependencies.

Feedforward network

mainly composed of feedforward neural networks, it receives input from the encoder module and outputs the final target sequence.

Attention Unit Design

Scaling dot product note

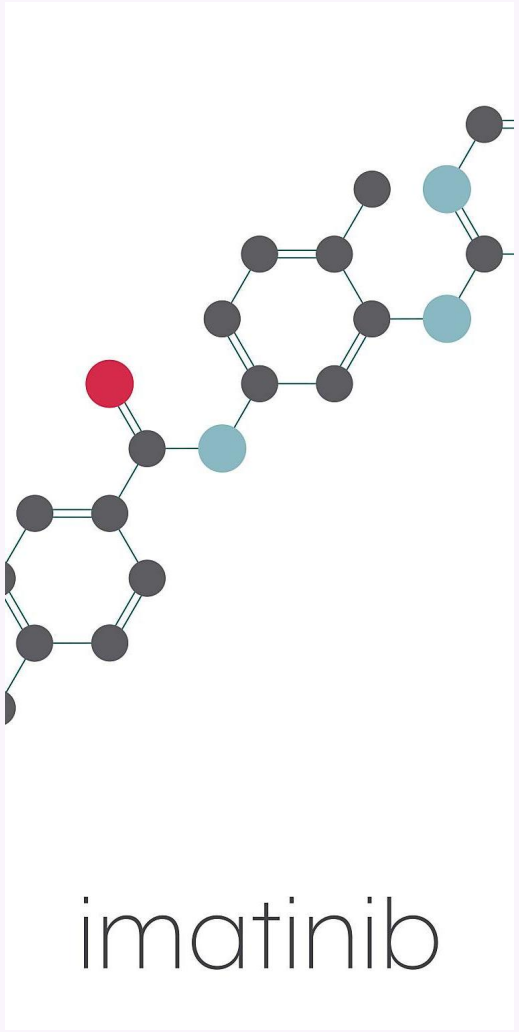
Calculate the dot product of the query and key, divide it by the scaling factor √ d_k, and then apply the softmax function to obtain the attention weight.

Multi head attention

By processing multiple self attention functions in parallel, independently calculating the attention of each position, and then merging the results, the model's processing capability and adaptability are enhanced.

Position encoding

Introducing position encoding allows the model to utilize position information and better process information from different positions in the input sequence.



04

Transformer operation process

Input vectorization processing

01

Tokenization

Convert input text into a sequence of numbers, discard or replace characters that cannot be processed, ensuring that the model can process them efficiently.

02

Vocabulary

Build a vocabulary that includes all possible words, convert the text into an index in the vocabulary, and digitize the text.

03

Embedding

Convert the word indices in the vocabulary into fixed length embedding vectors for easier processing and understanding of the model.

04

Handling Special Tokens

Design reasonable processing strategies for special characters such as periods and commas, such as converting them into specific embedding vectors or ignoring them.

Working principle of encoder layer

Self Attention Mechanism

calculates the similarity between each element in the input sequence and all other elements, generates attention weights, and automatically captures the relationships between elements.

Feedforward Network

By adding non-linear transformations through fully connected layers, the representations output by the self attention mechanism are further processed to learn more complex features.

Skip Connections

Within the encoding layer, residual connections are used to add the output and input of sub layers, effectively alleviating the problem of gradient vanishing and promoting information flow and model training.

Layer Normalization

Use layer normalization before and after sub layers to accelerate the model training process, stabilize network fluctuations, and improve model generalization ability.

Decoder layer working mechanism

- Masked Multi Head Attention: By using masks to limit the scope of self attention mechanism, the decoder is forced to only see the current position and previous information, ensuring that the decoding process follows a left to right order.
- Cross Attention: Connecting the encoder and decoder to enable the decoder to obtain the output information of the encoder, achieving information transmission and interaction between the encoder and decoder.
- Fully Connected Layer: Linearly transform the output of the cross attention mechanism to generate the final output sequence, and convert the output of the attention mechanism into the probability distribution predicted by the model through the linear layer.
- Decoder sub layer enhancement: By introducing sub layer enhancements such as Layer Normalization, Dropout, etc., the robustness and generalization ability of the decoder sub layer are improved, effectively alleviating overfitting problems.

05

Detailed explanation of location coding

Detailed Explanation of Position Encoding Technology

Positional Encoding

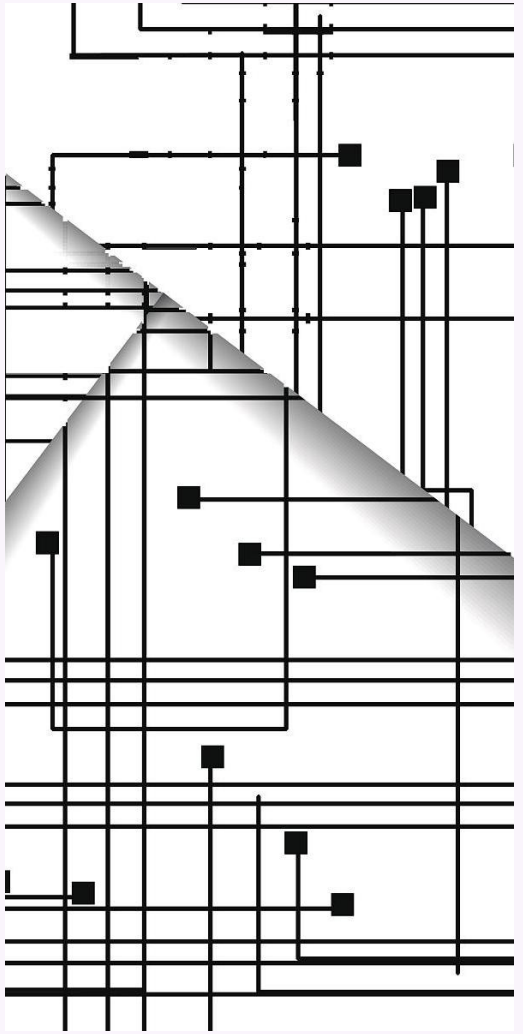
Due to the inability of models such as Transformers to directly understand sequence order, positional information needs to be injected into the model through positional encoding to assist in processing ordered data.

Techniques for Generating Positional Encoding

covering sine cosine encoding, learnable encoding, and relative position encoding, each method has unique advantages and applicable scenarios.

Embedding with Positional Encoding

Cleverly integrating positional encoding with embedding vectors, endowing the model with the ability to recognize different positional information in the sequence, and improving the efficiency of processing ordered data.



The principle of sine cosine coding

Encoding principle

Using sine and cosine functions, calculate the encoding value based on the position. The sine function is used for odd positions, and the cosine function is used for even positions, ensuring that each position is encoded uniquely.

Frequency selection

By combining sine and cosine functions of different frequencies, rich positional information is provided to the model, enhancing the understanding of element positions in the sequence.

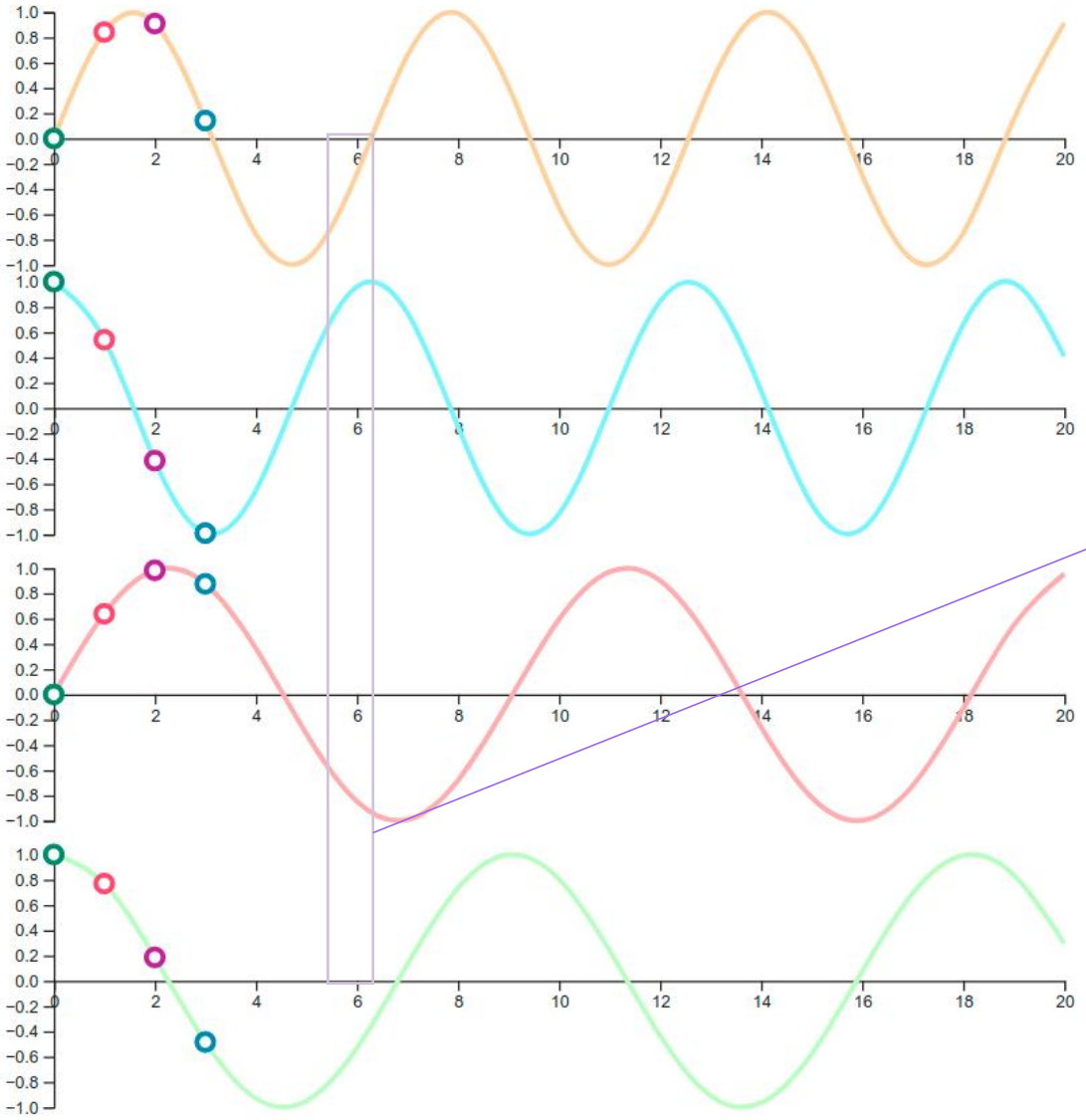
Infinite sequence support

Based on the encoding method of sine and cosine functions, theoretically it can support infinite length sequences without worrying about sequence length limitations, and has better scalability.

Unsupervised learning

Position encoding does not require special learning during model training, reducing model complexity, improving training efficiency, and avoiding overfitting risks.

Positional Encoding



p0	p1	p2	p3	
0.000	0.841	0.909	0.141	i=0
1.000	0.540	-0.416	-0.990	i=1
0.000	0.638	0.983	0.875	i=2
1.000	0.770	0.186	-0.484	i=3

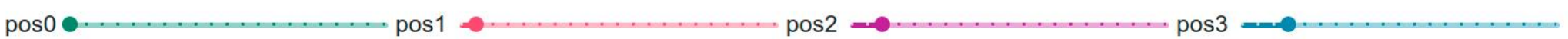
Positional Encoding

$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

Settings: d = 50

The value of each positional encoding depends on the *position* (*pos*) and *dimension* (*d*). We calculate result for every *index* (*i*) to get the whole vector.



Positional Encoding

Example： “The quick brown fox jumps.”

Positional Encoding

$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

- "quick" - position 1
- "brown" - position 2
- "fox" - position 3
- "jumps" - position 4

2. For each position, we will compute a positional encoding vector. If d_{model} is 4, we have two sine and two cosine functions with different wavelengths:

- $PE_{(0,0)} = sin(0/10000^{0/4}) = sin(0) = 0$
- $PE_{(0,1)} = cos(0/10000^{0/4}) = cos(0) = 1$
- $PE_{(0,2)} = sin(0/10000^{2/4}) = sin(0) = 0$
- $PE_{(0,3)} = cos(0/10000^{2/4}) = cos(0) = 1$
- Positional Encoding for "The" = [0, 1, 0, 1]

Necessity of location information

Sequential directionality

For Transformer models in NLP tasks, positional encoding is crucial as it gives the model the ability to handle sequential data directionality, enabling the model to distinguish the order of words.

Location aware

By introducing location encoding, the model can explicitly perceive information from different locations, thereby more effectively processing sequence data with specific orders, such as text, time series, etc.

Encoding method selection

Relative position encoding enhances the model's perceptual ability through relative position information, enabling the model to better understand the relative relationships between elements in the sequence.

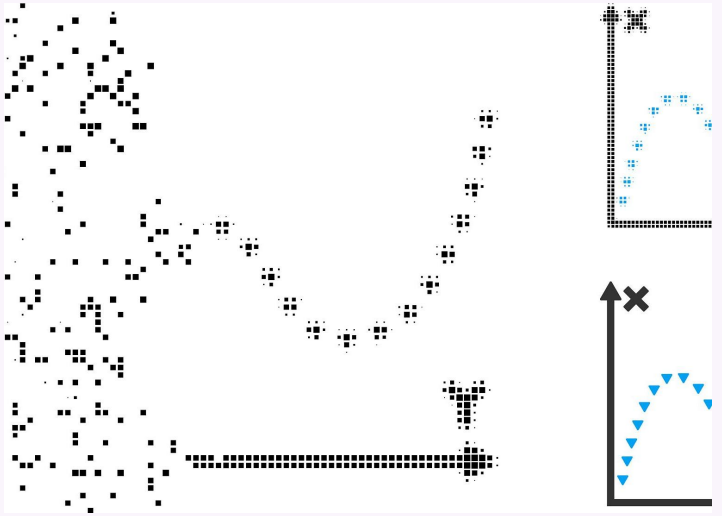
Improved performance

Relative position encoding enhances the performance of the model in processing sequence data by introducing relative position information, enabling the model to more accurately capture key information in the sequence.

Pay attention to the implementation method of masking

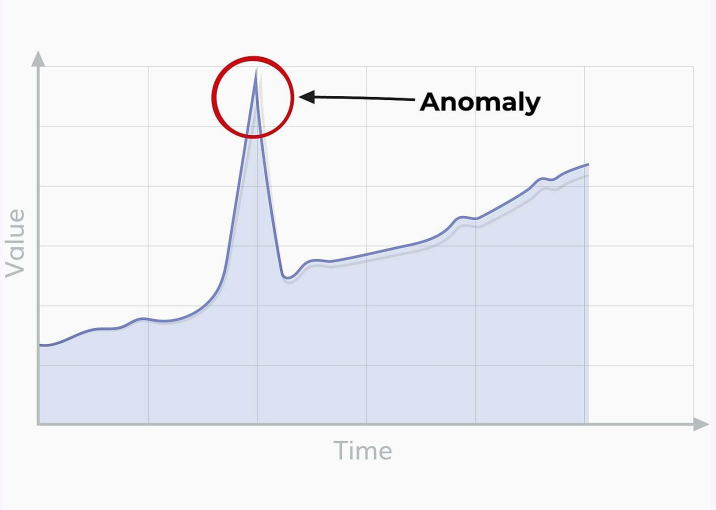
Masking in Self Attention

By setting a mask matrix, specific positions are ignored in self attention calculations to ensure that the model focuses on the relevant parts of the current task.



Implementation Strategy

Build specific masking strategies based on specific tasks (such as NSP training for BERT), accurately control information flow, and optimize model performance.



Customization

For complex scenes, flexibly adjust the shape and position of masks to ensure efficient and targeted model learning, and adapt to diverse needs.

Application scenarios of sequence mask

Control Information Flow

In BERT and other models, sequence masks are used to precisely control the flow of information, ensuring that the model focuses on the key information of the current task.

Implement Specific Tasks

such as BERT's MLM training, which designs specific mask patterns to achieve efficient word prediction and enhance the model's understanding of language structures.

Fine Tune for Diverse NLP Tasks

The sequence masking strategy can be fine tuned according to specific NLP tasks (such as sentiment analysis, question answering systems) to optimize model performance and adapt to different scenario requirements.

06

Multi-head Attention

Self Attention Calculation Process

Transform Vector

Transform each element (usually a word or subword) of the input sequence into an embedding vector, which will be used for subsequent attention calculations.

Calculate similarity

For each element in the input sequence, calculate its similarity to all other elements, usually measured using dot product or bilinear function.

Scaling similarity

In order to stabilize the training process, the calculated similarity is scaled, and the scaling factor is usually the square root of the sequence length.

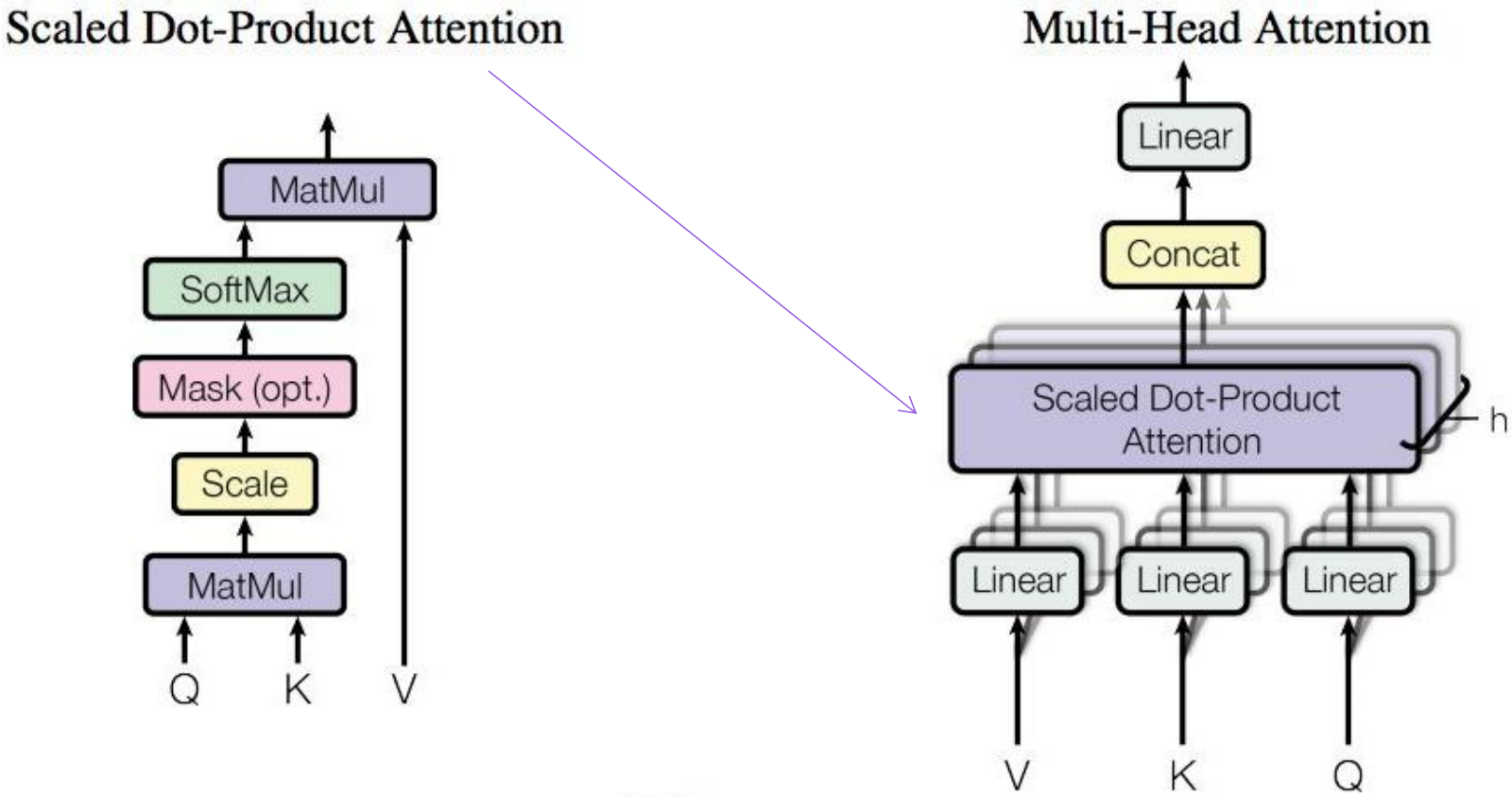
Apply softmax

Apply the softmax function to the scaled similarity to obtain the probability distribution of each position, which represents the attention weight.

Weighted sum

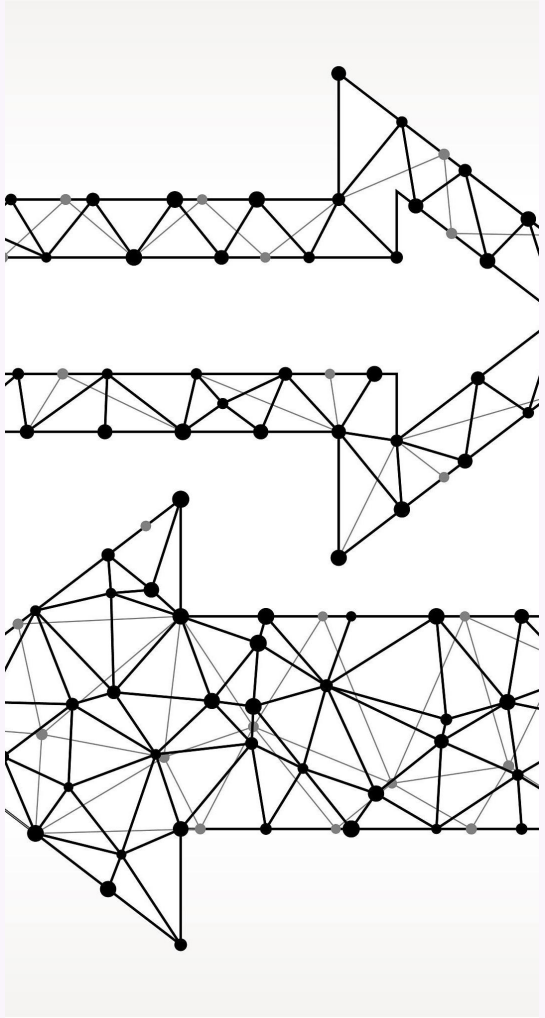
The output of the self attention layer is obtained by weighting each element in the input sequence with its corresponding attention weight, which will serve as the input for subsequent layers.

Multi-head attention (Self Attention)



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Principle of QKV Matrix Operation



Projection operation

The model transforms each element in the input sequence into a query, key, and value vector through three independent linear transformations (projection matrices).

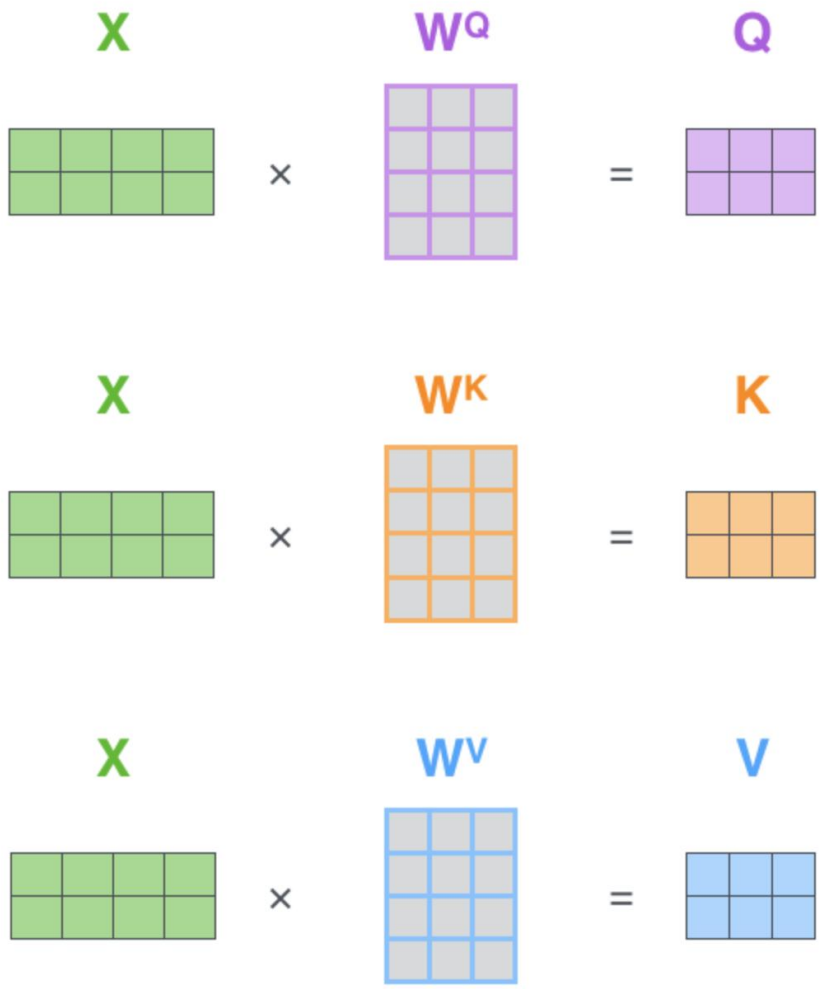
Efficient matrix operation

Query, key, and value vectors are processed through dot product operation to efficiently calculate attention scores. Additionally, it supports broadcast mechanism, allowing different queries to be matched with corresponding keys.

Information aggregation

Normalize scores through softmax activation function to obtain attention weights, which are used to aggregate value vectors and generate the final output representation.

Self-Attention : QKV Computation

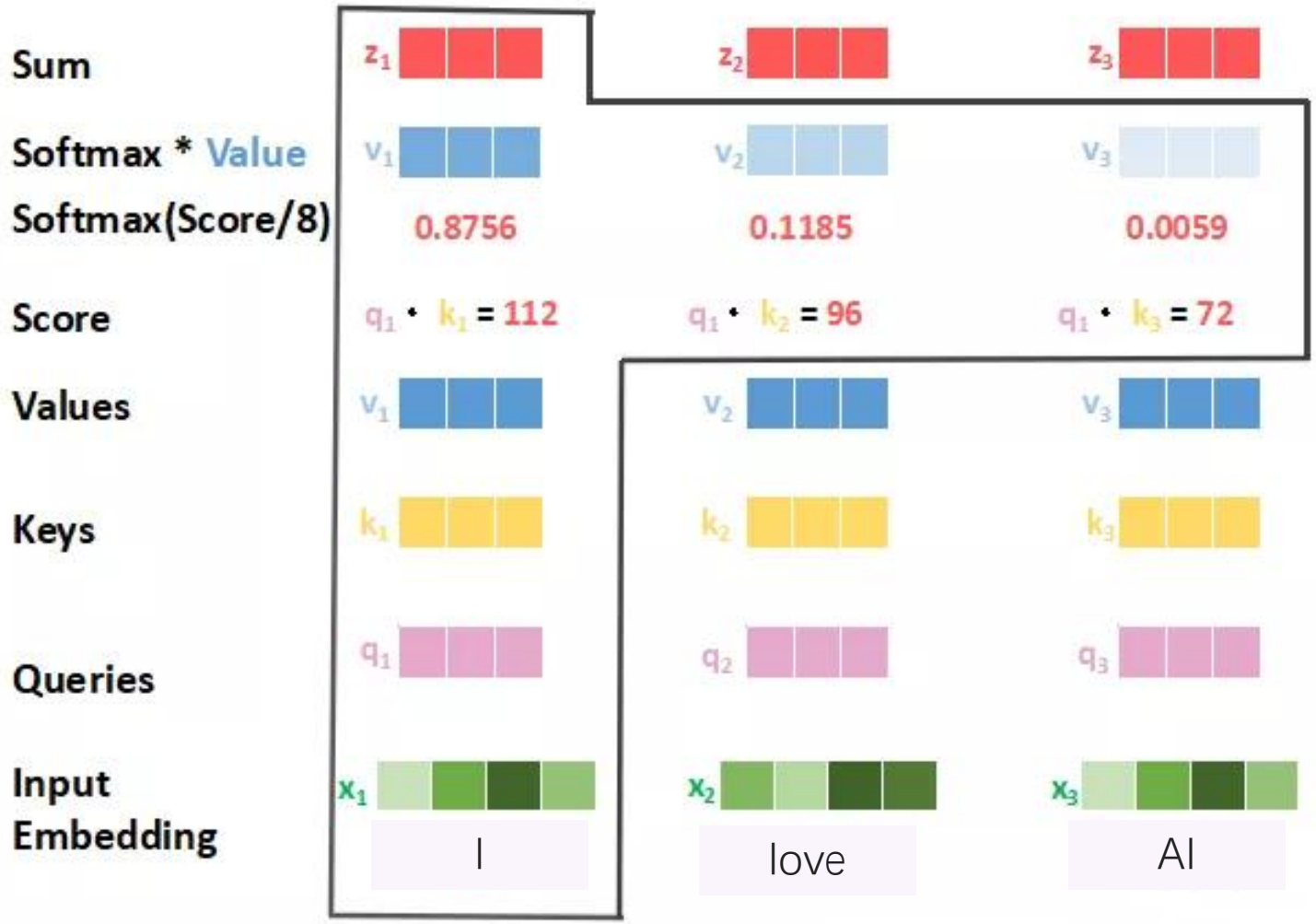


Self-Attention : QKV Computation

- These three matrices can be trained.
- The x vector has a dimension of 512, while q, k, and v each have a dimension of 64, resulting in a combined dimension of 512*64 for all three matrices.
- This is determined by the number of grids.



Self-Attention Example



07

Advantages and disadvantages of Transformer

Advantages of Parallel Processing

- Multi head self attention mechanism: The Transformer model adopts a multi head self attention mechanism, which can efficiently process parallel input sequences, significantly improving processing speed and efficiency. Traditional RNNs models are limited in parallelization.
- Scalability: The parallel processing capability of Transformer makes it easy to scale and cope with large-scale datasets and complex models, effectively improving model performance and accuracy, bringing broad prospects for deep learning applications.
- Efficient resource utilization: Through parallel processing, Transformers can more efficiently utilize computing resources and reduce time complexity, which is particularly significant when processing large-scale data and helps accelerate model training and inference processes.
- Widely applicable: The parallel processing capability of Transformer makes it suitable for various tasks such as machine translation, image recognition, speech recognition, etc., demonstrating its wide applicability and powerful capabilities, injecting momentum into the development of the AI field.

Limitations of Long Sequence Processing

- Long sequence challenge: Transformer models may face challenges when dealing with very long sequences. As the sequence length increases, computational complexity and memory requirements sharply rise, which may lead to performance degradation or resource scarcity.
- Gradient vanishing/exploding: In deep learning, long sequence processing may suffer from gradient vanishing or exploding problems, which can affect model training. Transformers alleviate through attention mechanisms, but long sequences still pose challenges.
- Limitations: Despite Transformer's powerful representation capabilities, its performance may not be as good as RNN models such as LSTM or GRU when dealing with tasks that require long-term dependencies, such as text generation.
- Improvement direction: To address the challenge of long sequences, research focuses on developing more efficient Transformer architectures, optimizing training algorithms, and utilizing external memory mechanisms to enhance model performance and overcome existing limitations.

Computing resource requirements

- High resource requirements: Transformer models require high computing resources and memory due to their complex structure and large-scale data processing capabilities. This includes acceleration hardware such as GPU/TPU that supports large-scale matrix operations.
- Training cost: Due to the high complexity of the model, the training cost of Transformer is usually very expensive, requiring a large amount of power and computing resources. Optimizing training algorithms and hardware utilization are key to reducing costs.
- Inference resources: When deploying Transformer models for inference, resource requirements also need to be considered. Edge devices and mobile devices typically have limited resources and require specific optimization of the model for deployment.
- The necessity of optimization: In response to the challenge of high resource demand, model optimization has become crucial. This includes reducing model size, using more efficient algorithms and data structures, and utilizing hardware acceleration to improve performance.

8

Analysis of BERT Model

Characteristics of bidirectional encoder

Bidirectional encoding

BERT adopts a bidirectional encoder structure, which can simultaneously process the contextual information of the sequence, providing comprehensive contextual support for the task.

Deep optimization

Through the design of multi-layer Transformer encoders, BERT is able to capture more complex language structures and semantic associations.

Pre training application

BERT is built on a foundation of extensive pre training and can adapt to specific tasks such as question answering, sentiment analysis, etc.

Rich Embedding

Use rich embedding layers to capture word, position, and sentence boundary information, enhancing the model's understanding of language details and structure.

Innovation in Pre training Methods

01

Mask language model

Randomly masking some input vocabulary, forcing the model to recover these masked words from the context, enhancing language understanding.

02

Next sentence prediction

By predicting the content of the next sentence in the article, the model can learn to understand the relationships and reasoning between sentences, improving overall text comprehension.

03

Unsupervised learning

Adopting unsupervised learning methods, training with a large amount of unlabeled data, reducing dependence on labeled data, and improving model generalization ability.

04

Fine tuning and optimization

When fine-tuning specific NLP tasks, BERT can quickly adapt and improve task performance, such as sentiment analysis, question answering systems, etc.

Application in NLP

Emotion analysis

Using the BERT model to analyze the emotional color of text, such as positive, negative, or neutral, to enhance customer experience and product improvement.

Question and Answer System

Build a BERT based question and answer system that can answer user questions and provide accurate and relevant answers.

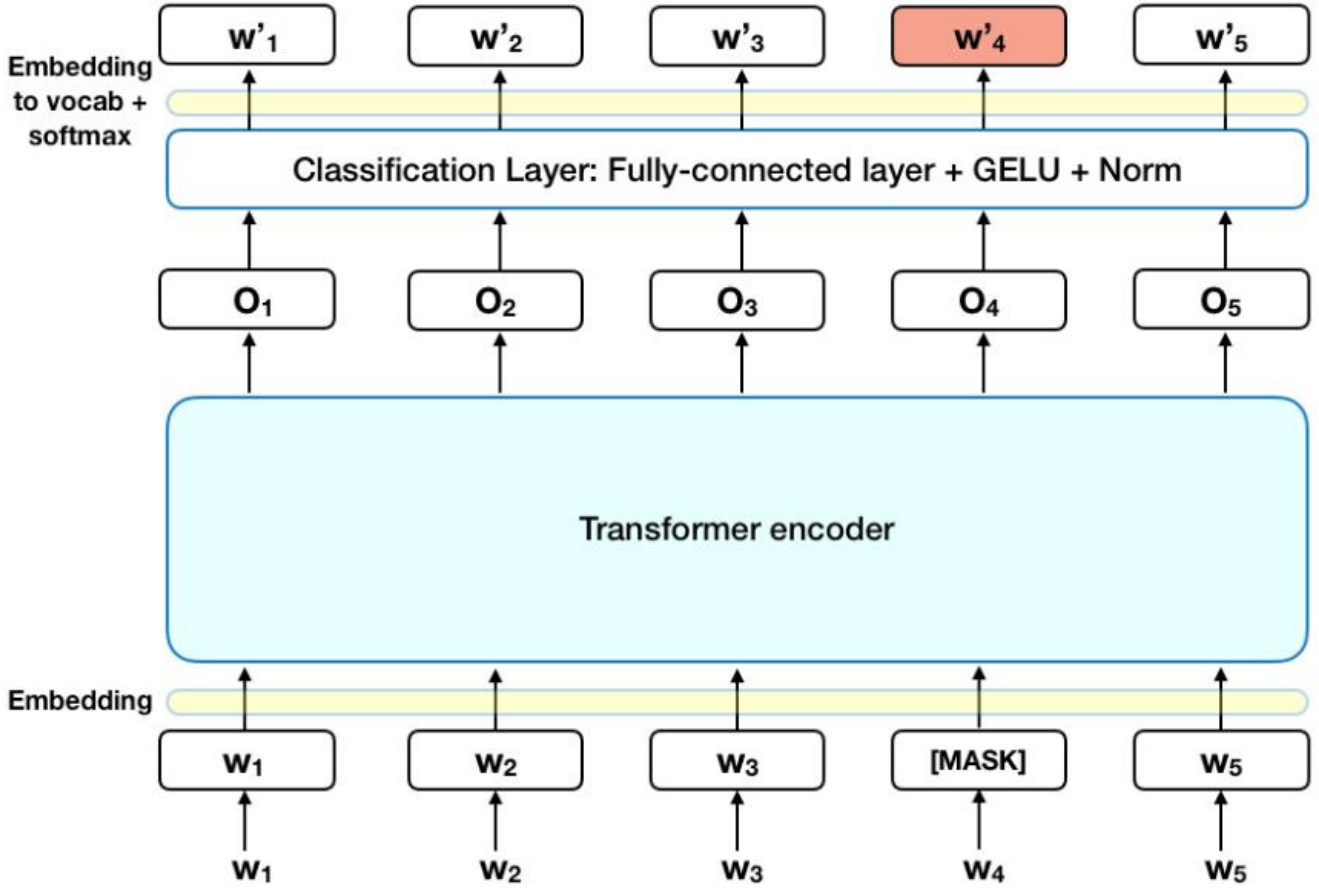
Machine translation

With the encoding capability of BERT, text from one language is converted into another language to achieve efficient and accurate machine translation.

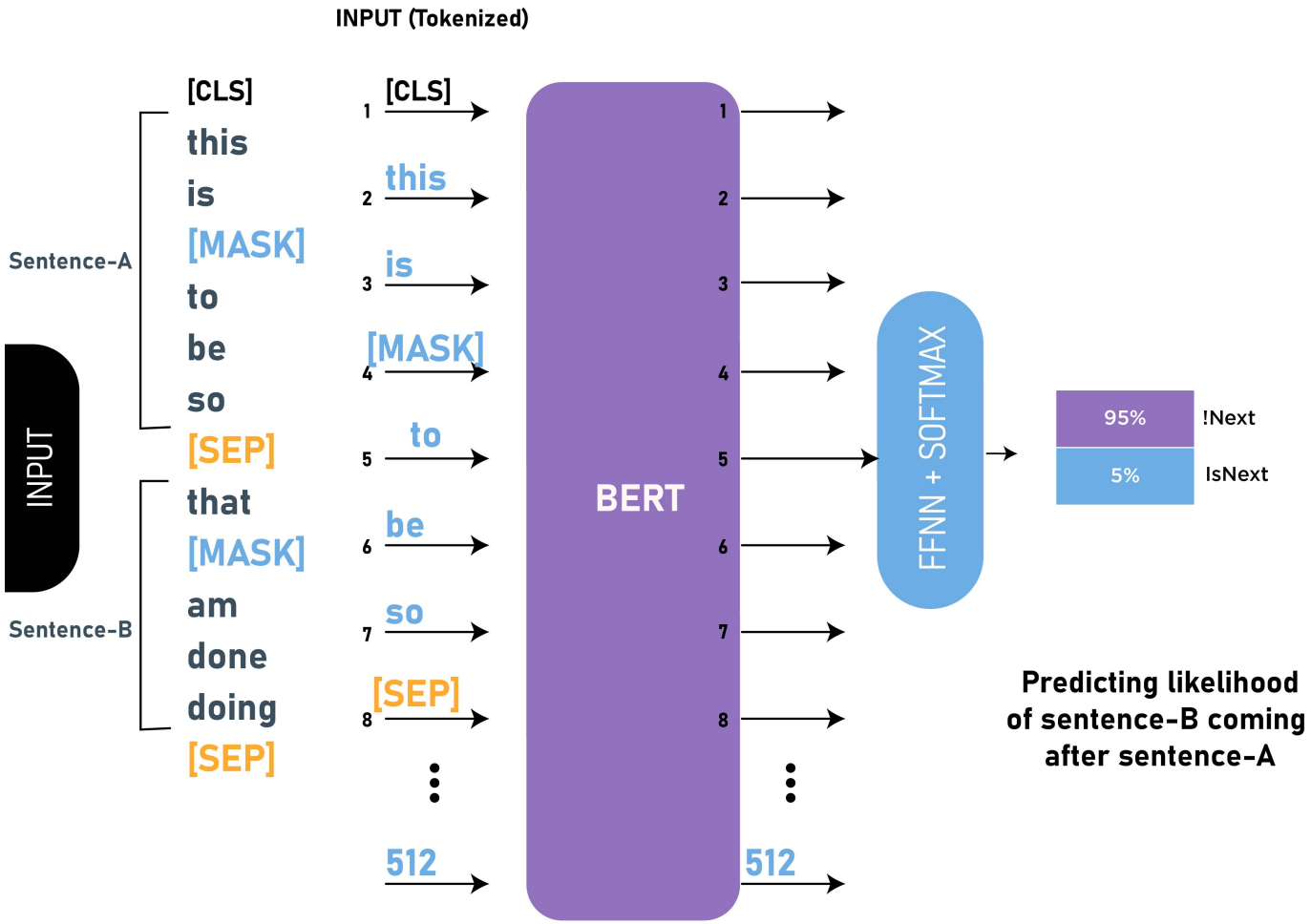
Text generation

Using BERT to generate coherent and fluent text content, such as abstract generation, story creation, etc., demonstrating powerful text generation capabilities.

BERT Architecture



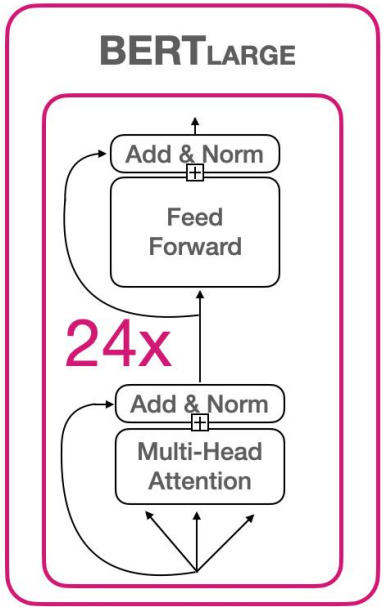
BERT Example



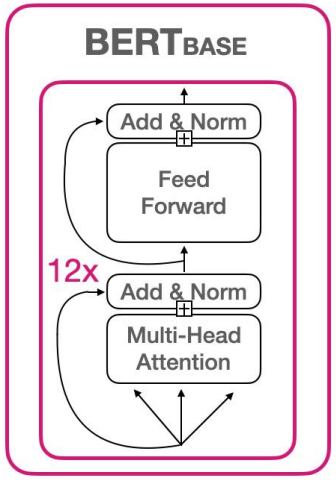
BERT Model Size

- BERT base 110M parameters.
  16 TPU, 4 days of training
  (no hyperparameter tuning).

- BERT large 340M parameters.
  64 TPU, 4 days of training
  (no hyperparameter tuning).



340M Parameters



110M Parameters

BERT Embedding

Token Embeddings

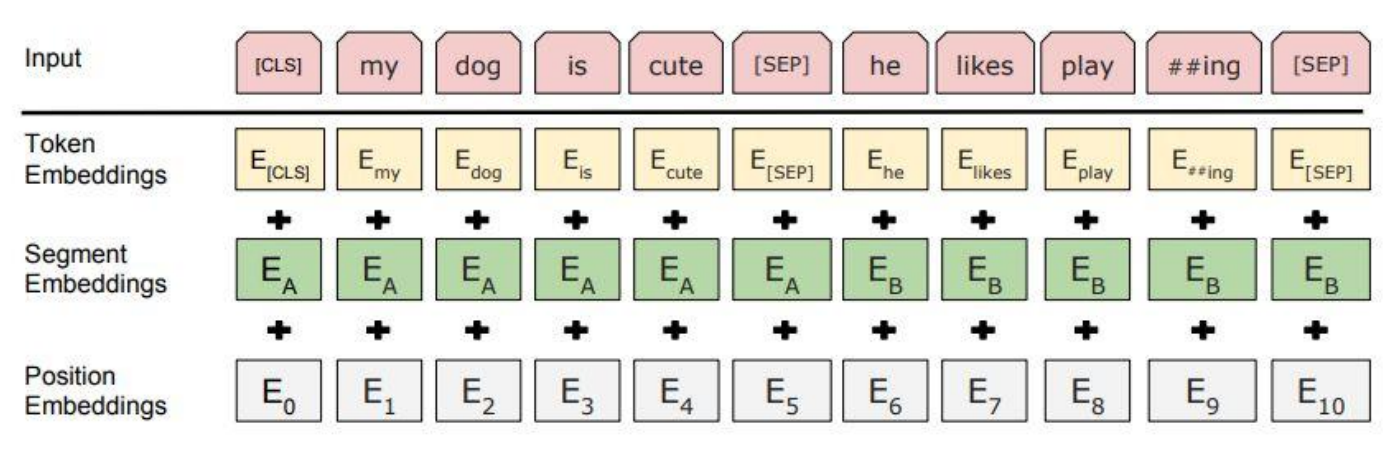
- 使用 BERT 分词器将输入文本转换为整数分词 ID 列表

Segment Embeddings

- 用于区分序列中的两个句子

Position Embeddings

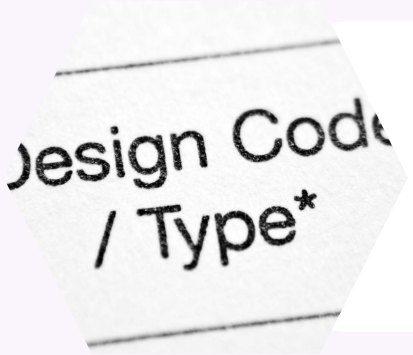
- 用于对序列中每个标记的位置进行编码



9

BERT Training Method

Mask Language Model (MLM)



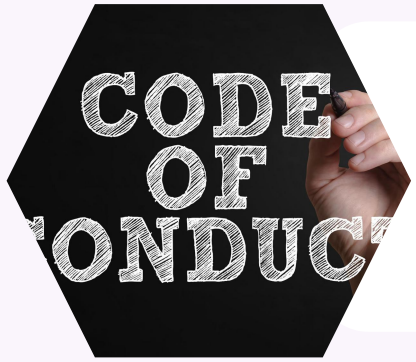
MLM task

Randomly mask certain parts of the input text, allowing the model to learn language representations when predicting these masked parts. Through this task, the model is able to understand the importance of words and phrases.

Mask application

Mask certain words in the input text by replacing them with special tags. The model needs to predict these masked words in order to learn how to generate reasonable sentences.

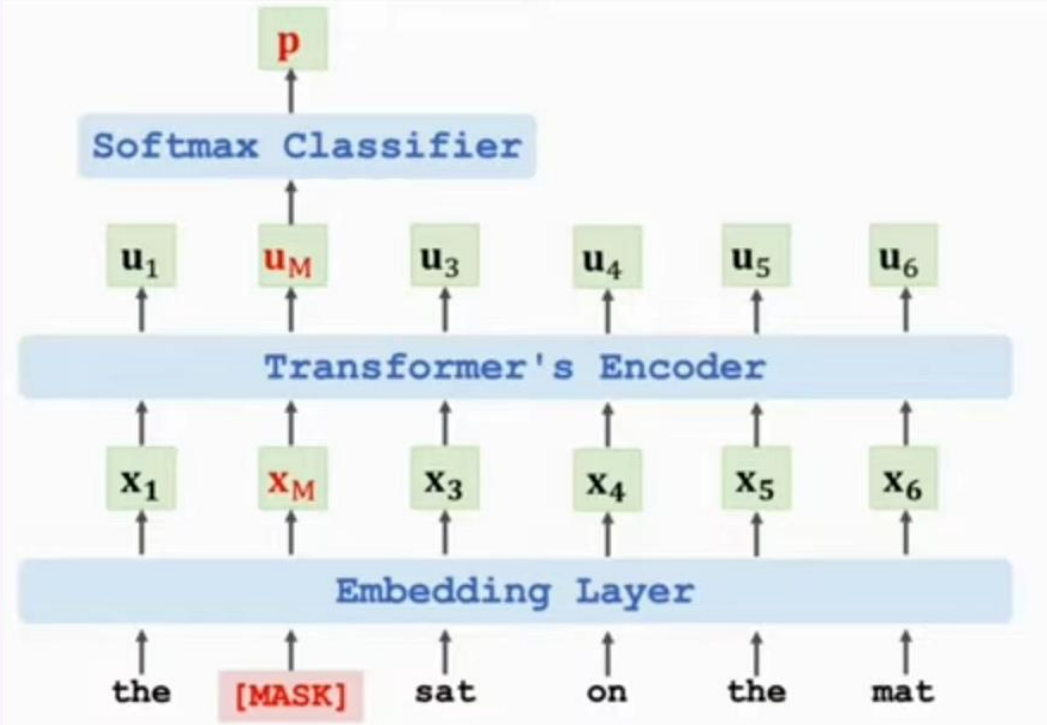




Language comprehension

Through MLM tasks, the model can learn how to understand the meaning of words and phrases in a given context. This helps improve the performance of language models in natural language processing tasks.

Mask Language Model (MLM)



- **e**: one-hot vector of the masked word "cat".
- **p**: output probability distribution at the masked position.
- $\text{Loss} = \text{CrossEntropy}(\mathbf{e}, \mathbf{p})$.
- Performing one gradient descent to update the model parameters.

Next Sentence Prediction (NSP)

NSP task

Determine whether two text fragments should be arranged consecutively. Through this task, the model can learn how to understand the relationships between sentences and infer which sentences are relevant.

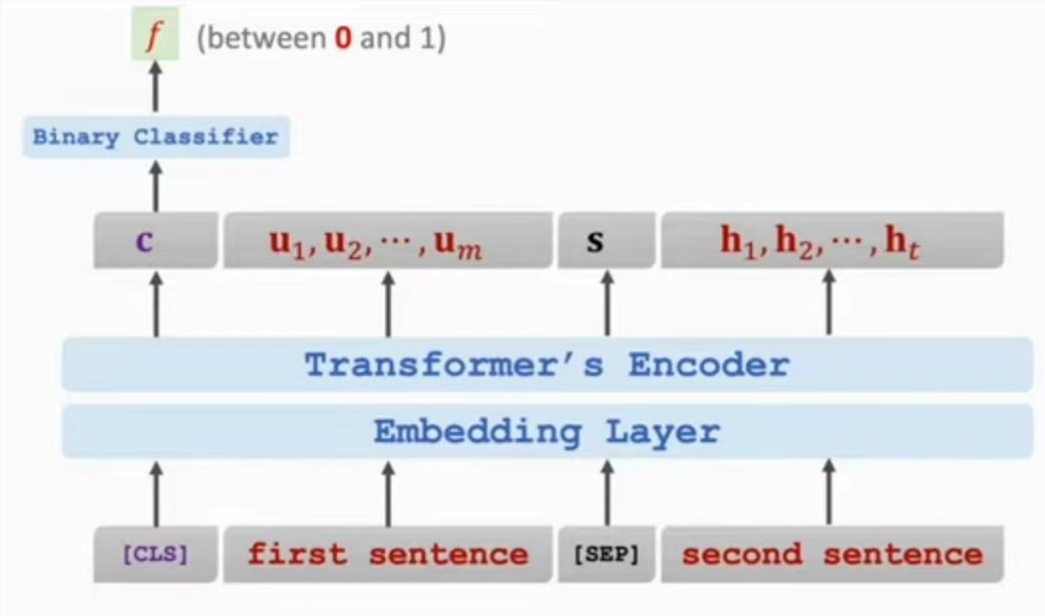
Sentence association

Given a sentence pair, the model needs to predict whether these two sentences will appear consecutively in the text. This helps the model understand the coherence and correlation between sentences.

Understanding reasoning

Through NSP tasks, the model can learn how to infer relationships between sentences based on their content and context. This helps improve the language model's ability to handle reasoning tasks.

Next Sentence Prediction (NSP)



Input:
[CLS] "calculus is a branch of math"
[SEP] "it was developed by newton and leibniz"

Target: true

Input:
[CLS] "calculus is a branch of math"
[SEP] "panda is native to south central china"

Target: false

Fine tune parameter settings



Fine tuning preparation

Load the pre trained BERT model and prepare for fine-tuning. This stage involves adjusting the general language representation to a language representation for specific tasks, such as sentiment analysis or question answering systems.

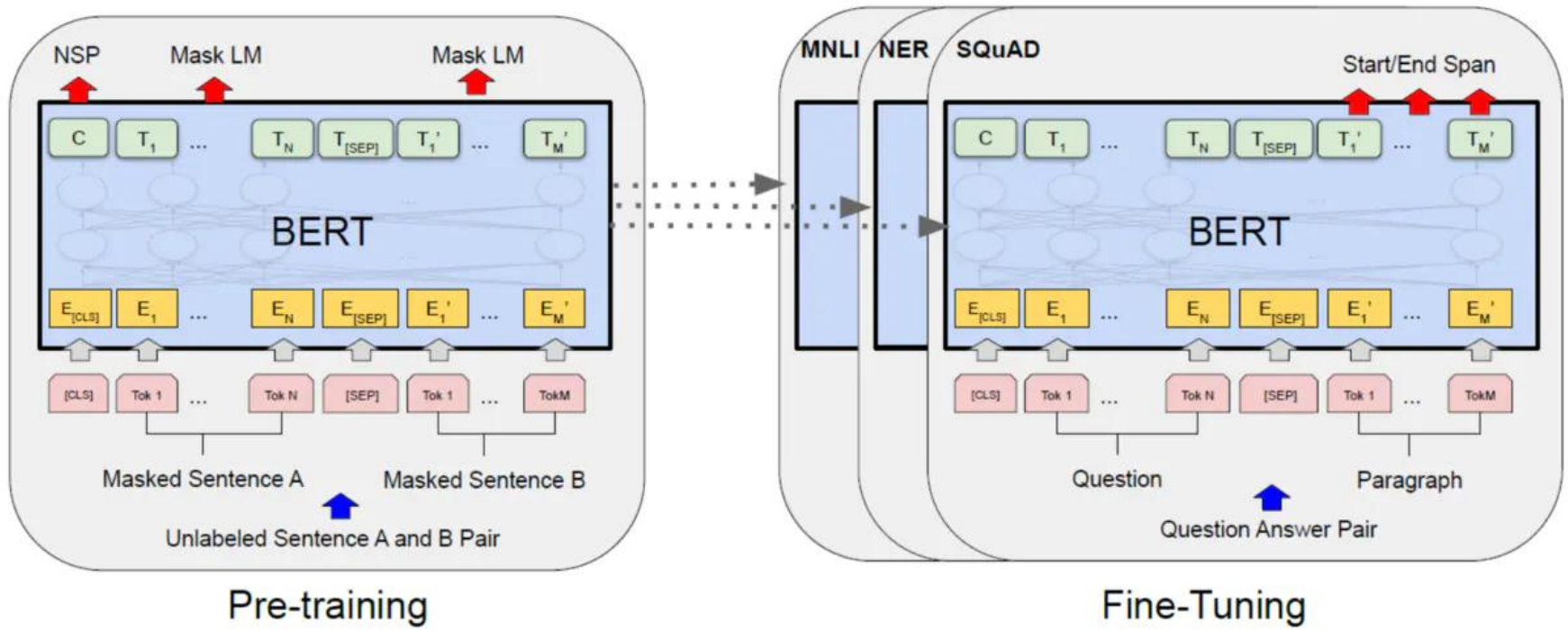
Fine tuning process

Fine tune the parameters of the BERT model to adapt to specific tasks. This includes adjusting the size of the embedding layer, hidden layer, as well as attention and feedforward network parameters.

Fine tuning objective

Fine tuning aims to optimize model performance and improve the accuracy of the model on specific tasks through training data. After fine-tuning, the model can better adapt to new tasks.

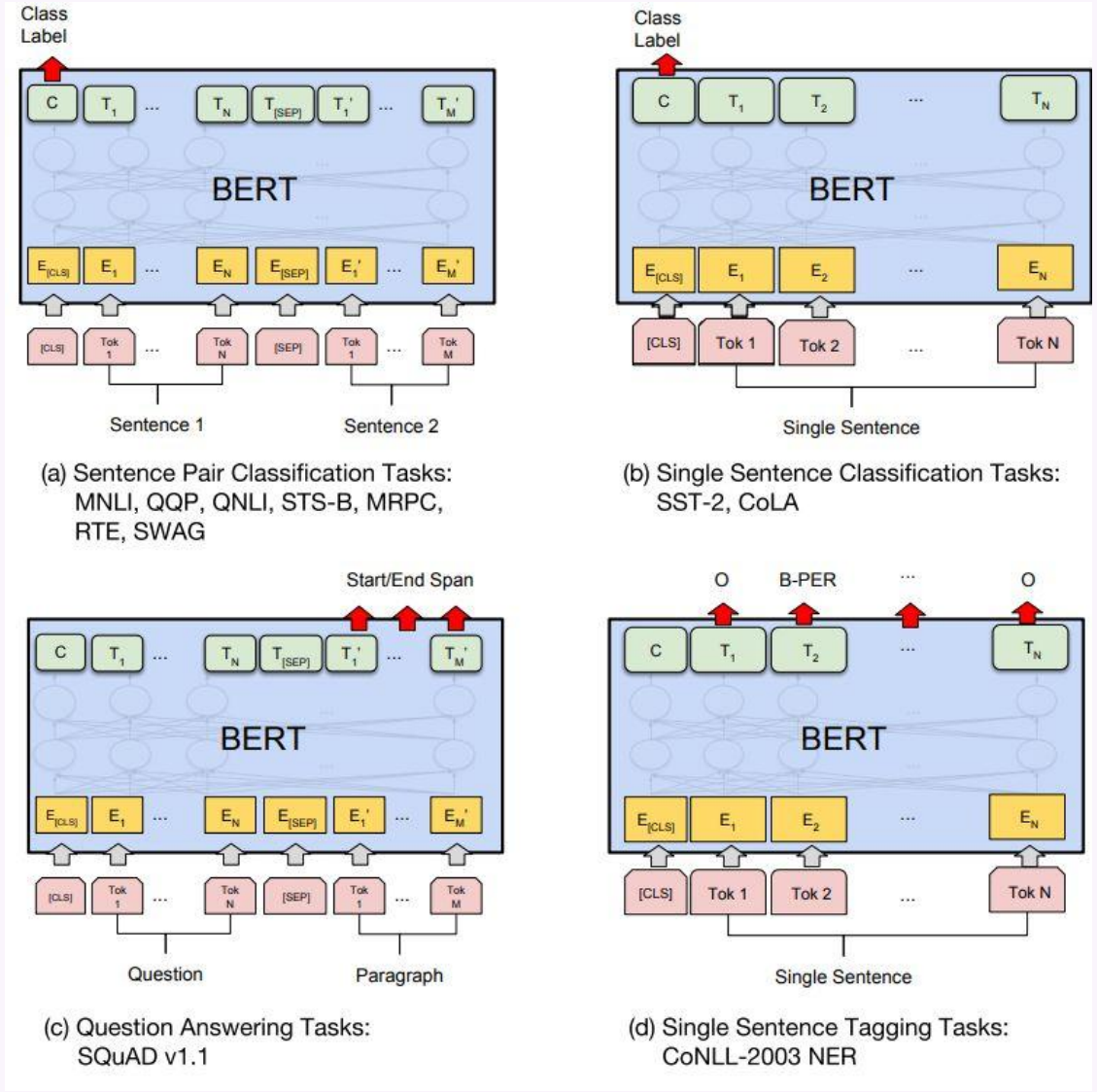
NLP 应用程序的微调


NSP
Mask LM
Mask LM
BERT
Masked Sentence A
Masked Sentence B
Unlabeled Sentence A and B Pair
Pre-training
MNLI
NER
SQuAD
Start/End Span
BERT
Question
Paragraph
Question Answer Pair
Fine-Tuning

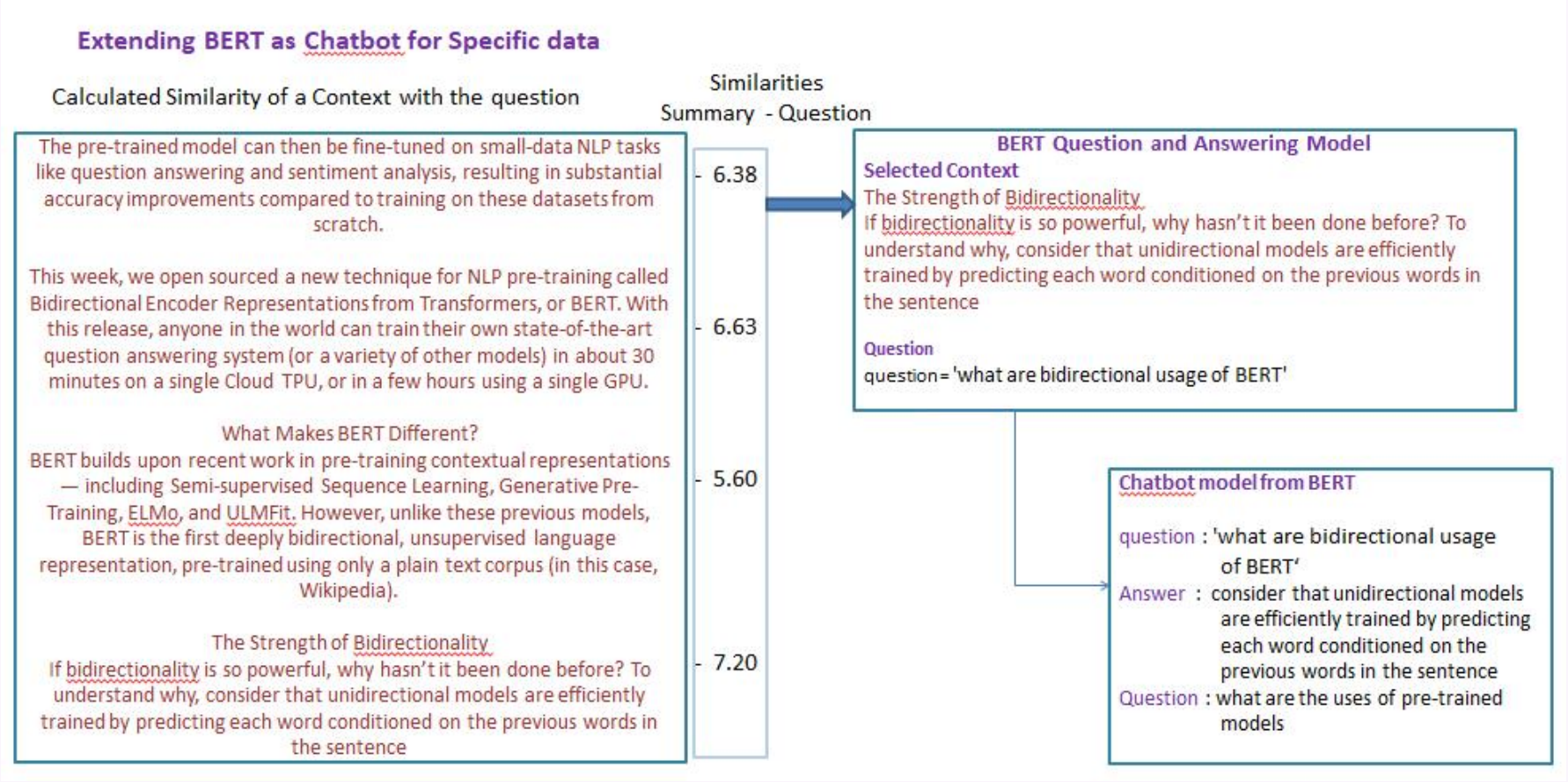
NLP 应用程序的微调

The following parameters and values can be adjusted:

- Batch size: 16, 32
- Learning rate (Adam): 5e-5, 3e-5, 2e-5
- Number of epochs: 3, 4



BERT Customized Chatbot



BERT Chatbot with Pytorch Code

```
from transformers import BertTokenizer, BertForSequenceClassification
from torch.utils.data import DataLoader, Dataset
import torch
import torch.nn.functional as F
# Sample dataset class
class ChatDataset(Dataset):
    def __init__(self, sentences, tokenizer, max_len=128):
        self.sentences = sentences
        self.tokenizer = tokenizer
        self.max_len = max_len
    def __len__(self):
        return len(self.sentences)
    def __getitem__(self, item):
        sentence = str(self.sentences[item])
        encoding = self.tokenizer.encode_plus(
            sentence,
            add_special_tokens=True,
            max_length=self.max_len,
            return_token_type_ids=False,
            padding='max_length',
            return_attention_mask=True,
            return_tensors='pt',
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten()
        }
```

```
# Load pre-trained model and tokenizer
model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForSequenceClassification.from_pretrained(model_name)

# Example sentences
sentences = ["Hello, how can I help you?", "Goodbye!"]
dataset = ChatDataset(sentences, tokenizer)

def chatbot_response(model, tokenizer, sentence):
    model.eval()
    with torch.no_grad():
        inputs = tokenizer.encode_plus(sentence, return_tensors="pt", add_special_tokens=True)
        input_ids = inputs["input_ids"]
        attention_mask = inputs["attention_mask"]

        output = model(input_ids, attention_mask=attention_mask)
        prediction = torch.argmax(F.softmax(output.logits, dim=1), dim=1)
        return prediction

# Example of getting a response
response = chatbot_response(model, tokenizer, "Hello, who are you?")
print(f"Response ID: {response.item()}")
```

10

Transformer variants

Transformer Models Comparison

Model	Year	Key Characteristics
Transformer	2017	Introduced self-attention, sequence-to-sequence without RNNs
BERT	2018	Bidirectional context, pre-trained Transformer encoder on large corpus
GPT	2018	Unsupervised pre-training, Transformer decoder
Transformer-XL	2019	Recurrence mechanism, long context windows
Reformer	2020	Locality-sensitive hashing, process very long sequences
T5	2020	Text-to-text task framework, versatile
XLNet	2019	Autoregressive, bidirectional context
ALBERT	2019	Parameter-reduction techniques, scalable
ELECTRA	2020	Sample-efficient pre-training task
DeBERTa	2021	Disentangled attention, enhanced position encodings
MABA Transformer	Recent	Batch-level processing, master-branch attention

Different attention mechanisms

- Multi head attention mechanism: The standard multi head attention mechanism captures various relationships at different positions in the input sequence by processing multiple attention heads in parallel, enhancing the model's ability to model complex dependency relationships.
- Customized multi head attention: By adjusting the number and dimension of heads, as well as introducing different types of attention mechanisms (such as local attention), the model can more flexibly adapt to the needs of different sequence lengths.
- Self attention variant: The self attention mechanism allocates attention weights by calculating the similarity between queries, keys, and values, achieving automatic alignment and aggregation of information at different positions in the input sequence.
- Connection attention mechanism: Combining multi head attention and connection attention, the model can automatically learn and integrate global and local information, improve the understanding and processing ability of complex sequence structures, and promote NLP task performance.

Positional encoding variant

01

Sine coding

Standard sine coding utilizes sine and cosine functions of different frequencies to generate unique codes for each position in the sequence, which remain unchanged during model training.

02

Learnable encoding

learnable encoding generates unique encoding for each position in the sequence through parameters learned during the training process, which can capture more complex patterns.

03

Relative encoding

Relative encoding does not generate fixed encoding for each position in the sequence, but rather relative positional information. This encoding method is more flexible and can better adapt to different sequence lengths.

04

No encoding

Some variants completely omit positional encoding and rely on self attention mechanisms to capture positional information in sequences, learning the importance of positions through training.

Improvement of Feedforward Network

- Activation function: The standard Feedforward network mainly uses the ReLU activation function to enhance the model's nonlinear expression ability. But in order to better capture the distribution characteristics of data, more complex activation functions such as GELU can be chosen.
- Dropout rate: By adjusting the Dropout dropout rate, the model can balance the risks of overfitting and underfitting. The standard configuration is 0.1, but for specific tasks, this value may need to be adjusted to optimize model performance.
- Normalization: Adding layer normalization to the Feedforward network helps stabilize the training process by reducing internal covariate bias, improving the model's generalization ability and training efficiency.
- Network structure: Some variants construct deeper Feedforward networks by adding intermediate layers or changing the size of layers to enhance the model's representational power. This deep design can better capture complex features.

Mixture of Experts

01

The concept of MOE

Definition of Mixture of Experts

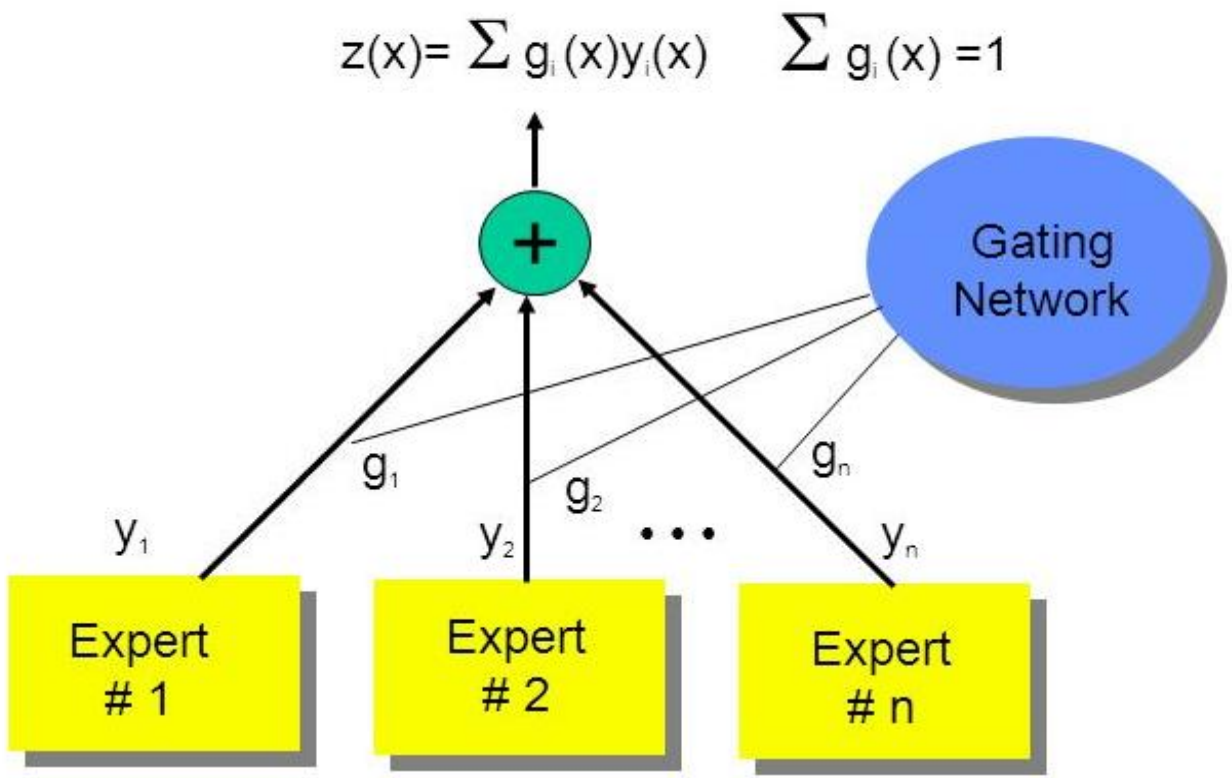
Mixture of Expert Model

MOE (Mixture of Experts) is a complex machine learning model that integrates multiple expert networks to collaborate in processing input data, achieving efficient and accurate prediction or classification.

Capacity Expansion with Experts

In a standard Transformer, MOE replaces a single feedforward network layer with multiple expert networks, with each expert focusing on a specific task, enhancing the model's processing power and flexibility, and promoting a win-win situation of efficiency and performance.

Mixture of Expert Network



Conditional computing system

Conditional Computation

POE implements conditional computation, dynamically activating expert subsets based on input features, non-traditional fixed network structure, flexible and efficient processing of diverse data, improving computational efficiency and model generalization ability.

Gating Network Routing

As the core component, the Gating Network intelligently evaluates and dynamically selects the most relevant expert set to ensure that each input is processed optimally, driving precise and efficient conditional computation.

Dynamic selection of subnetworks

Dynamic Subnet Selection

Input tokens are evaluated by a gated network, and weight allocation is used to determine whether experts are retained or not, achieving dynamic selection of subnets. This mechanism enhances the flexibility and adaptability of the model, efficiently responding to the challenges of changing data.

Top-K Expert Activation

Select Top-K experts, focus on high weight areas, and ignore low contributors. This strategy optimizes the utilization of computing resources, accelerates processing flow, and ensures that the model efficiently and accurately responds to complex and changing task challenges.

Weighted Sum Output

The final output is obtained by combining the outputs of the selected experts and weighted sum. The weights are precisely adjusted by the gating network to ensure that the contributions of each expert are reasonably integrated, thereby improving the overall model performance and prediction accuracy.

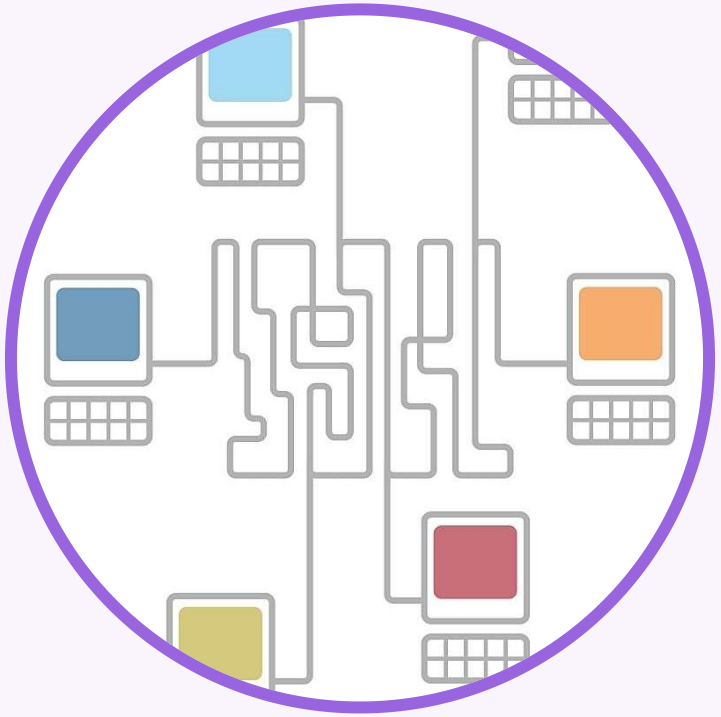
02

Core formula

Gate control weight calculation

Token input

Each input token vector is processed by a gated network, which includes linear layers and Softmax activation, generating weight vectors equal to the number of experts.



Noise addition

To enhance the model's exploration ability, adjustable noise can be introduced, which is optimized during the training process to balance model performance and exploration efficiency.

Weight calculation

The gate control weight matrix is combined with the input token vector to calculate the weight of each expert through matrix multiplication, achieving dynamic selection.

Gating Network (Router) Calculation:

For an input token representation $x \in \mathbb{R}^d$, the router computes a score (logit) for each expert j:

$$logits_j = x \cdot W_r^{(j)} + b_r^{(j)},$$

where $W_r \in \mathbb{R}^{E \times d}$ is the router's weight matrix and E is the total number of experts. These logits are converted into probabilities via a softmax function:

$$p_j = \frac{\exp(logits_j)}{\sum_{i=1}^{E} \exp(logits_i)}.$$

Top-K Expert Selection



Weight sorting

Sort experts based on their weights, select the K experts with the highest weights (usually K is much smaller than the total number of experts), and reset the remaining weights to zero.

Activate sparsity

By using the Top-K selection mechanism, only some experts are activated, significantly reducing computational costs, improving model efficiency, and avoiding overfitting risks.





Expert utilization

Top-K strategy ensures that each input token is only processed by the most relevant expert, enhancing the model's generalization ability and improving overall performance.

Top-K Selection and Weight Normalization:

Instead of using all experts, only the top K experts with the highest probabilities are selected (a common choice is $K = 2$). The probabilities for these selected experts are re-normalized to form a weight:

$$w_j = \frac{p_j}{\sum_{i \in top_K} p_i}, for j \in top_K.$$

Tokens that are not routed to any expert are typically passed through a residual connection.

Output weighted sum

Expert output

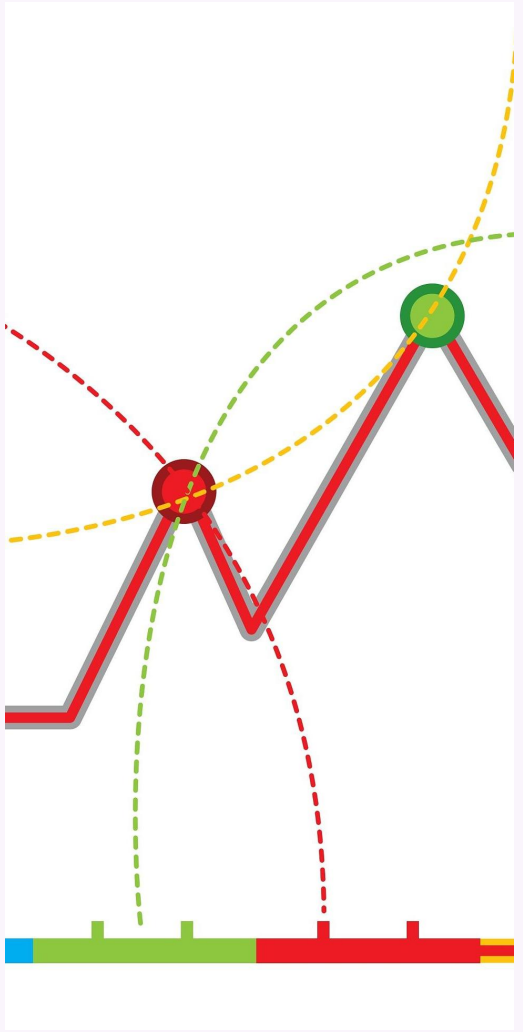
For the selected Top-K experts, calculate a weighted sum based on their output and corresponding weights. The FFN layer is usually used as the expert output layer to extract advanced features.

Output layer

The final output is the weighted sum of the selected expert outputs, which are fused through linear combination to form a more comprehensive representation and improve model performance.

Gradient update

During the training process, the parameters of the gating network and expert network are updated using backpropagation algorithm to optimize model performance and ensure accurate selection and utilization of experts.



Expert Output and Weighted Sum:

Each expert is an independent neural network (e.g., an FFN). The output of the MoE layer is the weighted sum of the outputs from the selected experts:

$$y = \sum_{j \in top_K} w_j \cdot Expert_j(x).$$

Auxiliary Load Balancing Loss (Optional but Crucial):

To prevent the router from always selecting the same few popular experts (a "rich get richer" problem), an auxiliary loss term is often added to the overall model loss to encourage more balanced expert utilization :

$$L_{aux} = \alpha \cdot E \cdot \sum_{j=1}^{E} f_j \cdot P_j,$$

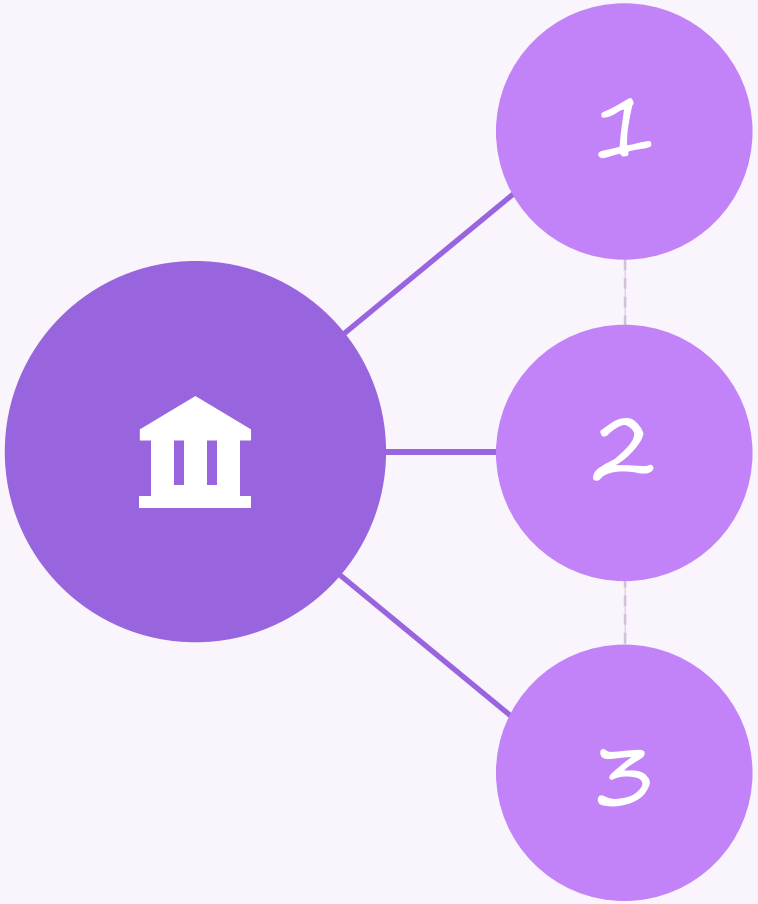
where:

- f_j is the fraction of tokens routed to expert j in a batch.
- P_j is the mean probability assigned to expert j by the router for all tokens in the batch.
- E is the total number of experts.
- α is a hyperparameter scaling the importance of this loss (e.g., 0.01).

03

Calculation example

Input vector processing



Input tokenization

The input vector is processed by a gating network to generate weights selected by experts. This step is the core of the MOE mechanism, determining which experts will participate in the calculation of the current input.

Weight calculation

Based on the input token, the gating network outputs a set of logits, which are processed by the Softmax function and converted into the selection weights of each expert to guide the expert selection process.

Top-K selection

Retain the Top-K experts with the highest weight, and reset the remaining weights to zero. This step ensures that only the most relevant experts participate in the calculation, improving the efficiency and accuracy of the model.

MOE Example

Input token vector $x = [0.5, -0.2, 0.1, 0.3]$ (hidden dimension $d = 4$)

Number of experts $E = 4$, activate top $K = 2$

Router parameters:

Weight matrix $W_r = \begin{bmatrix} 0.2 & -0.1 & 0.0 & 0.1 \\ -0.3 & 0.4 & 0.2 & -0.2 \\ 0.1 & 0.0 & -0.2 & 0.3 \\ 0.0 & 0.2 & 0.1 & -0.1 \end{bmatrix}$

Bias $b_r = [0.0, 0.1, -0.1, 0.05]$

For simplicity, each "expert" is a linear transformation (a stand-in for a full FFN):

$$Expert_j(x) = x \cdot W_e^{(j)} + b_e^{(j)}$$

Let's define the experts' weight matrices (as diagonal matrices for simplicity):

$W_e^{(0)} = diag([0.1, 0.1, 0.1, 0.1])$, $b_e^{(0)} = [0, 0, 0, 0]$

$W_e^{(1)} = diag([0.2, 0.2, 0.2, 0.2])$, $b_e^{(1)} = [0, 0, 0, 0]$

$W_e^{(2)} = diag([0.05, 0.05, 0.05, 0.05])$, $b_e^{(2)} = [0, 0, 0, 0]$

$W_e^{(3)} = diag([-0.1, -0.1, -0.1, -0.1])$, $b_e^{(3)} = [0, 0, 0, 0]$

Expert output calculation

Expert output calculation

Assuming two experts (Expert $\propto$ and Expert $_{0}$) are selected, each expert is an independent neural network structure that receives the same input vector and generates output.

Expert calculation differences

There may be differences in the output calculations of experts, for example, Expert $\propto$ may have used ReLU activation functions, while Expert $_{0}$ may have used other types of activation functions.

Output integration

The final model output is a weighted sum of the outputs of the selected experts. Through this approach, the model can utilize the knowledge of multiple experts to improve the accuracy and reliability of decision-making.

Step 1 : Compute Router Logits:

$$logits = x \cdot W_r^{\top} + b_r$$

$$= [0.5, -0.2, 0.1, 0.3] \cdot \begin{bmatrix} 0.2 & -0.3 & 0.1 & 0.0 \\ -0.1 & 0.4 & 0.0 & 0.2 \\ 0.0 & 0.2 & -0.2 & 0.1 \\ 0.1 & -0.2 & 0.3 & -0.1 \end{bmatrix} + [0.0, 0.1, -0.1, 0.05]$$

$$= [0.15, -0.27, 0.12, -0.01] + [0.0, 0.1, -0.1, 0.05] = [0.15, -0.17, 0.02, 0.04]$$

Step 2 : Compute Probabilities via Softmax:

$$p = softmax([0.15, -0.17, 0.02, 0.04]) \approx [0.289, 0.210, 0.254, 0.247]$$

The probabilities for experts 0, 1, 2, and 3 are ~0.289, 0.210, 0.254, and 0.247, respectively.

Weighted combination process



Weighted combination

After obtaining the output of each expert, the model will multiply these outputs with their corresponding weights and sum them up to obtain the final model output, reflecting the contributions of each expert.



Weight function

Weights play a crucial role here, determining the impact of each expert's output in the final decision, ensuring that only the most relevant experts provide information.



Improving model accuracy

Through weighted combination, the model can more effectively integrate the knowledge of various experts, thereby improving the accuracy and reliability of decision-making.

Step 3 : Select Top-K Experts and Normalize Weights:

Top-2 experts are expert 0 (0.289) and expert 2 (0.254).

Normalize their weights:

$$w_0 = \frac{0.289}{0.289 + 0.254} \approx 0.532$$

$$w_2 = \frac{0.254}{0.289 + 0.254} \approx 0.468$$

Step 4: Compute Expert Outputs:

$$Expert_0(x) = [0.5, -0.2, 0.1, 0.3] \cdot diag([0.1, 0.1, 0.1, 0.1]) = [0.05, -0.02, 0.01, 0.03]$$
$$Expert_2(x) = [0.5, -0.2, 0.1, 0.3] \cdot diag([0.05, 0.05, 0.05, 0.05]) = [0.025, -0.01, 0.005, 0.015]$$

Step 5: Compute Final MoE Output:

$$y = w_0 \cdot Expert_0(x) + w_2 \cdot Expert_2(x)$$
$$= 0.532 \cdot [0.05, -0.02, 0.01, 0.03] + 0.468 \cdot [0.025, -0.01, 0.005, 0.015]$$
$$\approx [0.0266, -0.0106, 0.0053, 0.0160] + [0.0117, -0.0047, 0.0023, 0.0070]$$
$$\approx [0.0383, -0.0153, 0.0076, 0.0230]$$

04

Key Features

Key Characteristics

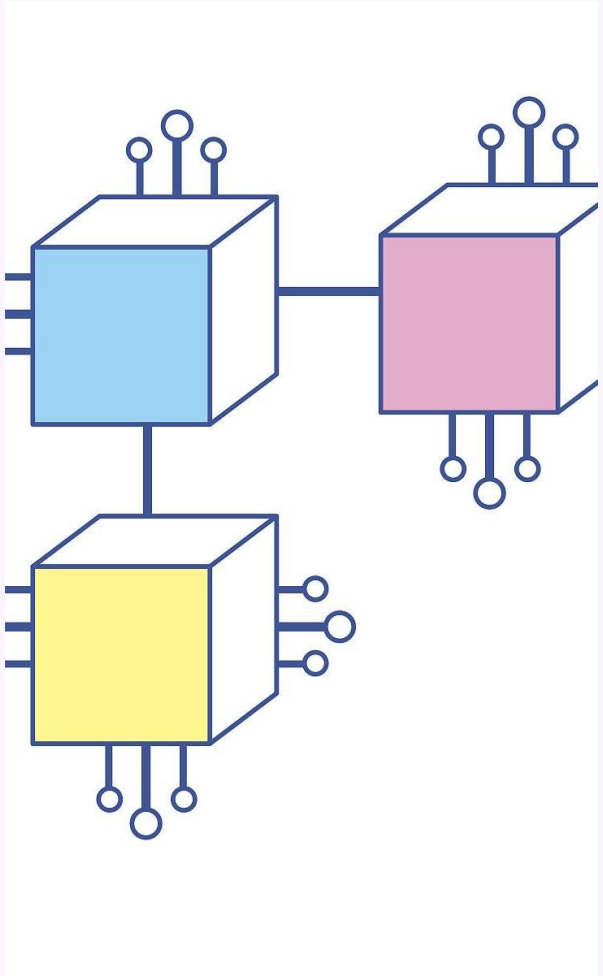
Sparsity and Efficiency: Computation is roughly proportional to $(K/E) \times (TotalParameters)$, not the total parameters themselves. For example, with $E = 8$ and $K = 2$, each token only uses 25% of the experts, saving significant computation .

Massive Scale: MoE enables the creation of models with trillions of parameters (e.g., Switch Transformer) that are computationally feasible to train and run during inference .

Challenges: Requires careful tuning to maintain load balancing and can have higher memory requirements for storing all experts. Communication costs can also be significant in distributed training setups.

This architecture is a foundational technique behind some of the largest and most powerful language models in use today, such as Mixtral 8x7B and Google's Switch Transformer .

Calculation efficiency advantage



Sparse activation reduces costs

MOE uses a sparse activation mechanism to select only a small number of experts to process inputs, significantly reducing computational costs and improving processing efficiency.

Expert selection is more efficient

each input token only activates the most relevant experts, resulting in a much lower total computation compared to traditional dense models, achieving efficient computation.

Optimization of computing resources

The computational efficiency advantage of MOE enables the model to run with limited computing resources, optimizing the use of computing resources.

Load balancing mechanism



Load balancing and hunger prevention

MOE effectively prevents expert hunger by introducing a load balancing mechanism, ensuring that each expert can participate in the calculation in a balanced manner.

Gated loss promotes balance



Load balancing regularization term is added to the gated loss function to guide the model to evenly use various experts and avoid uneven resource allocation.



Experts utilize balancing

Through load balancing mechanisms, MOE ensures that all experts can be fully utilized, improving the overall performance of the model.

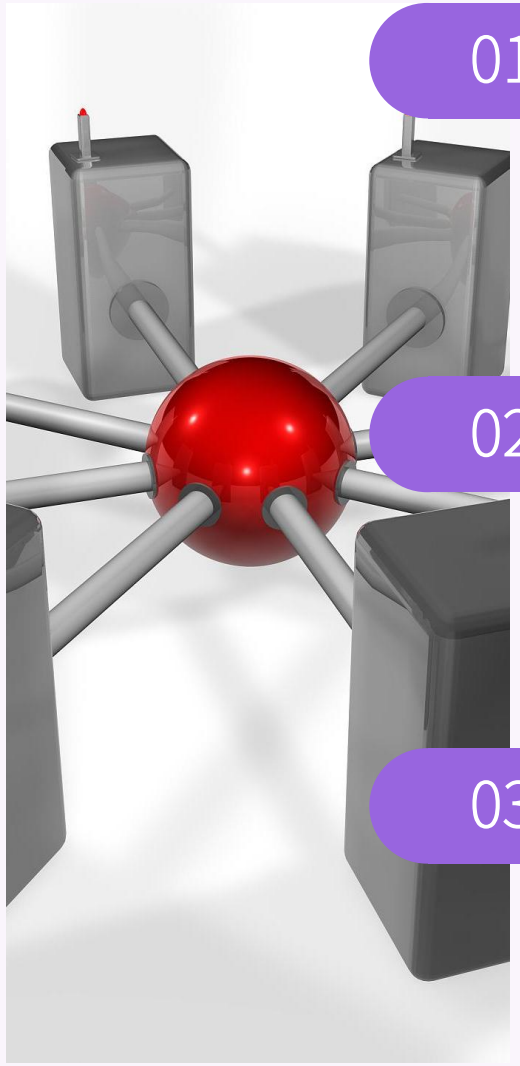
Large scale application scenarios

- Large scale model extension application: MOE can support the construction of large-scale language models by increasing the number of experts, achieving a significant expansion of parameter quantity, and improving model capacity.
- Efficient training and reasoning: MOE maintains low computational costs while expanding the number of parameters, achieving efficient training and reasoning and improving resource utilization efficiency.
- Multi task learning mastery: MOE utilizes the processing capabilities of multiple expert networks to effectively handle multi task learning scenarios and improve the performance of the model on different tasks.
- Domain adaptation enhancement: The dynamic selection mechanism of MOE enables the model to better adapt to data from different domains, enhancing the model's domain adaptability and generalization ability.

05

MOE Design in Transformer

MOE layer structure



01

Composition of Experts

In Transformer, the MOE layer significantly enhances the model's ability to handle complex tasks and significantly improves its expressive power by introducing multiple specialized FFNs (feedforward neural networks).

02

Gated mechanism

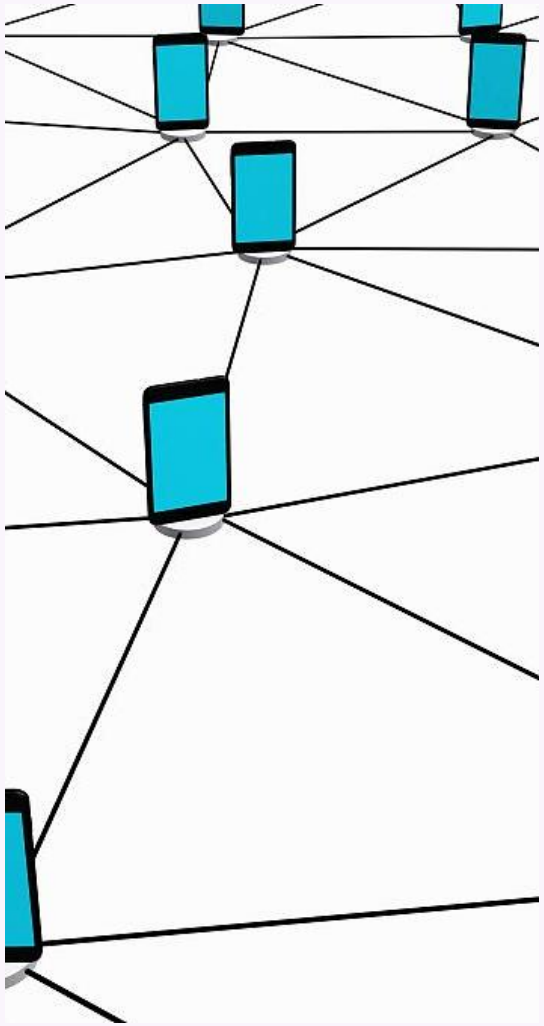
Gated networks play a crucial role in the MOE layer, dynamically selecting the most relevant subset of experts based on the characteristics of input data to ensure that each input token receives the most professional processing.

03

Output aggregation

The selected experts are multiplied by their corresponding weights and summed as the final output of the MOE layer. This process ensures that only the most relevant experts contribute to the output, improving the efficiency of the model.

Expert network composition



Independent Expert Network

Each expert is an independent FFN network, typically consisting of two linear layers with a non-linear activation function in between, such as ReLU (Rectified Linear Unit).

Selection of Expert Quantity

The number of experts is crucial in MOE design, ranging from tens to hundreds. For example, the GLaM model uses 137 experts to enhance the model's capacity and expressive power.

Load balancing

In order to prevent uneven load among experts, where some experts are overused while others are ignored, specific load balancing strategies need to be adopted to ensure that all experts can be fully utilized.

Gate control network function

01

Gated network selection

Gated networks receive input feature vectors and use linear layers and softmax (or hard selection) operations to assign each expert an "importance weight" to measure their relevance in processing the current input.

02

Number of activated experts

Based on importance weights, Top-K experts with the highest weights (such as Top-1 or Top-2) can be selected, and these experts will be activated to participate in the calculation of the current input, achieving dynamic expert selection.

03

Dynamic selection

The dynamic selection mechanism of the gate controlled network can automatically adjust the participating experts based on the characteristics of the input data, ensuring that only the most relevant experts contribute to the output and improving model efficiency.

06

Why does Transformer require MOE

Parameter expansion requirements

- The parameter quantity of Transformer: With the increase of model depth, the parameter quantity of Transformer grows exponentially, and the demand for hardware resources is also increasing, becoming a key factor restricting its application.
- MOE parameter expansion: By introducing the MOE mechanism, Transformer can efficiently expand the parameter quantity while maintaining low computational costs, providing possibilities for the implementation of large-scale language models and multimodal applications.
- The impact of increasing the number of experts: Increasing the number of experts can significantly improve the model capacity, but too many experts may lead to overfitting problems during the training process, which requires careful regulation to balance model performance and complexity.
- Balance between parameter and computational complexity: MOE achieves an effective balance between parameter and computational complexity by dynamically selecting some experts to process inputs, ensuring high performance of the model while reducing operating costs.

Optimization of computational efficiency

- Improvement of computational efficiency: The FFN layer in Transformer faces enormous computational pressure due to its dense connection structure, becoming a key factor restricting model efficiency.
- Optimization of MOE computational efficiency: By introducing the MOE mechanism, only a small number of experts are activated to process the input, significantly reducing the computational load and improving the inference speed of the model.
- Load balancing issue: In practical applications, there may be uneven loads among experts, which can affect the overall performance of the model. Corresponding measures need to be taken to achieve load balancing.
- Optimization strategy: Combining expert selection mechanism and load balancing technology, dynamically adjusting expert load to ensure balanced participation of experts in computation and improve overall model efficiency.

Improved adaptability to tasks

- Coping with task complexity: As task complexity increases, traditional Transformer models may face dual challenges of capacity and adaptability when dealing with new tasks.

- The adaptability advantage of MOE: By introducing multiple expert networks, MOE can flexibly adapt to the needs of different tasks and achieve wider applicability.

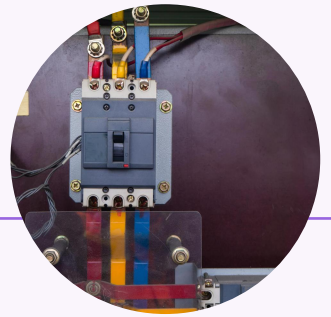
- Dynamic Matching of Gated Networks: Gated networks dynamically select relevant experts based on input features to ensure flexible switching between different tasks and improve task processing efficiency.

- Continuous optimization and exploration: Future research will further explore how to optimize the design of MOE, improve its ability to adapt to various complex tasks, and promote the development of the field of natural language processing.

07

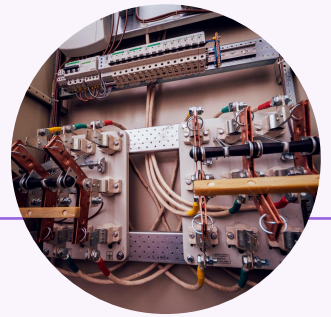
Typical MOE Transformer Model

SwitchTransformer



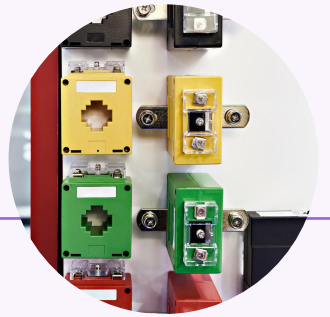
MoE debut

Switch Transformer innovatively introduces a hybrid expert mechanism, which is widely applied to Transformers to achieve efficient scaling.



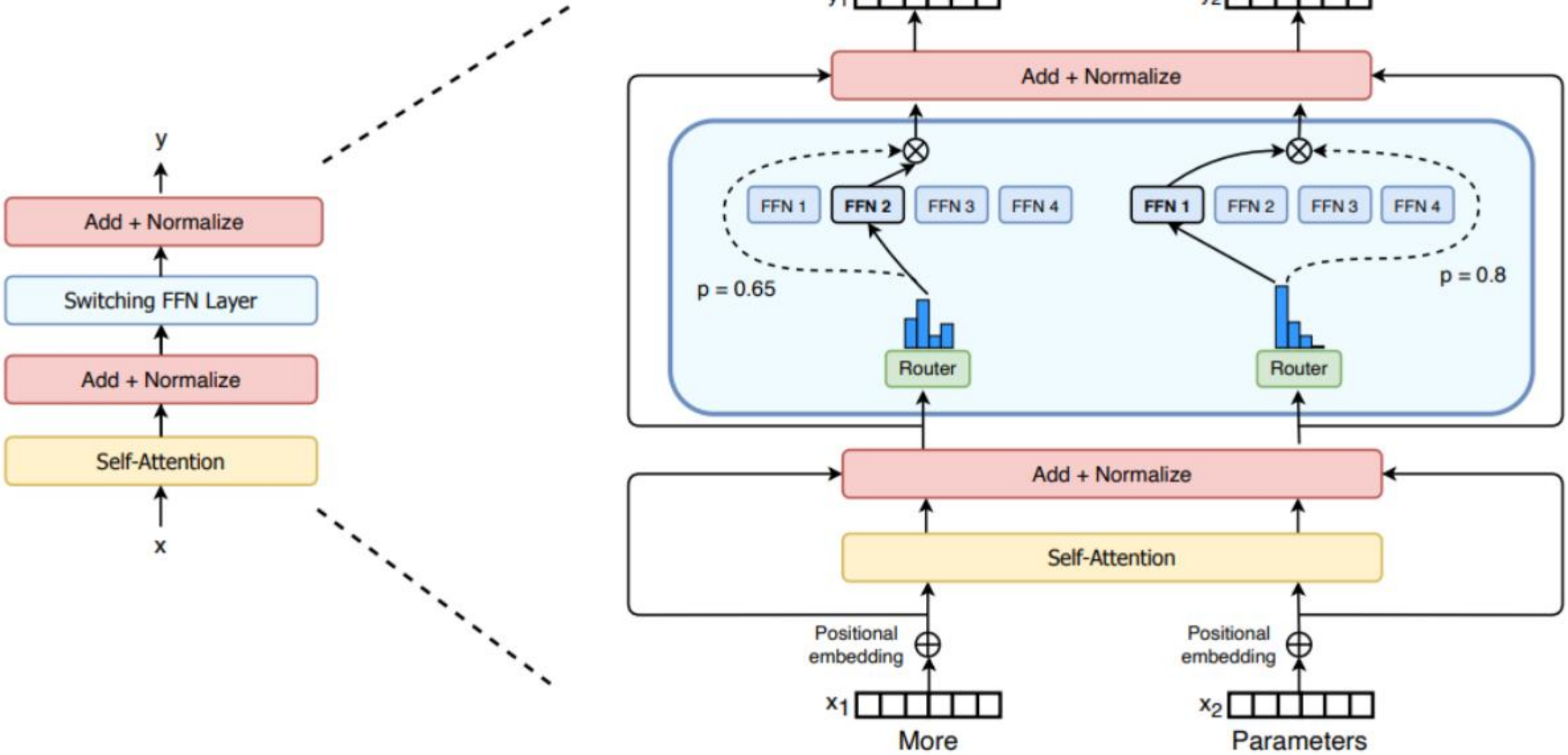
Ultimate scale

With a parameter count exceeding 1.6 trillion, the Top-1 gating strategy significantly improves training efficiency and demonstrates enormous model potential.



Performance leap

Google's masterpiece, Switch Transformer, uses sparse activation strategy to optimize resource utilization, resulting in a significant performance leap.


y
Add + Normalize
Switching FFN Layer
Add + Normalize
Self-Attention
x
y1
y2
Add + Normalize
FFN 1
FFN 2
FFN 3
FFN 4
FFN 1
FFN 2
FFN 3
FFN 4
p = 0.65
p = 0.8
Router
Router
Add + Normalize
Self-Attention
Positional embedding
Positional embedding
x1
x2
More
Parameters

GLaM （Generalist Language Model）

01

Multi expert collaboration

GLaM adopts Top-2 gating, gathers the wisdom of 137 experts, with parameters reaching 1.2 trillion, and jointly builds a new benchmark for NLP.

Beyond the baseline

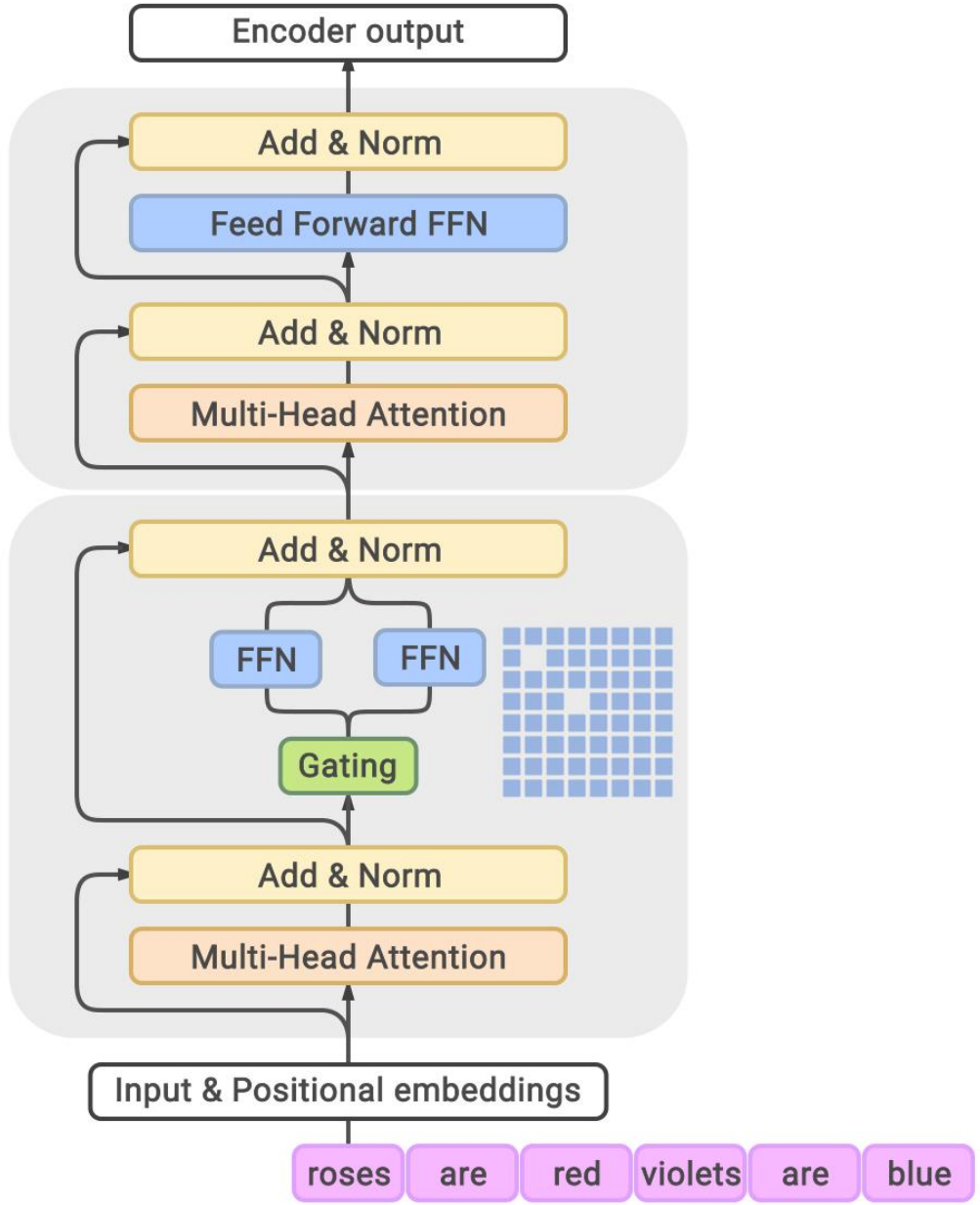
02

GLaM outperforms traditional dense transformers in various NLP tasks with its sparse activation strategy.

03

Efficiency and Performance

By dynamically selecting experts, GLaM achieves a dual improvement in performance and efficiency, demonstrating the advantages of a hybrid expert model.


Encoder output
Add & Norm
Feed Forward FFN
Add & Norm
Multi-Head Attention
Add & Norm
FFN
FFN
Gating
Add & Norm
Multi-Head Attention
Input & Positional embeddings
roses
are
red
violets
are
blue

08

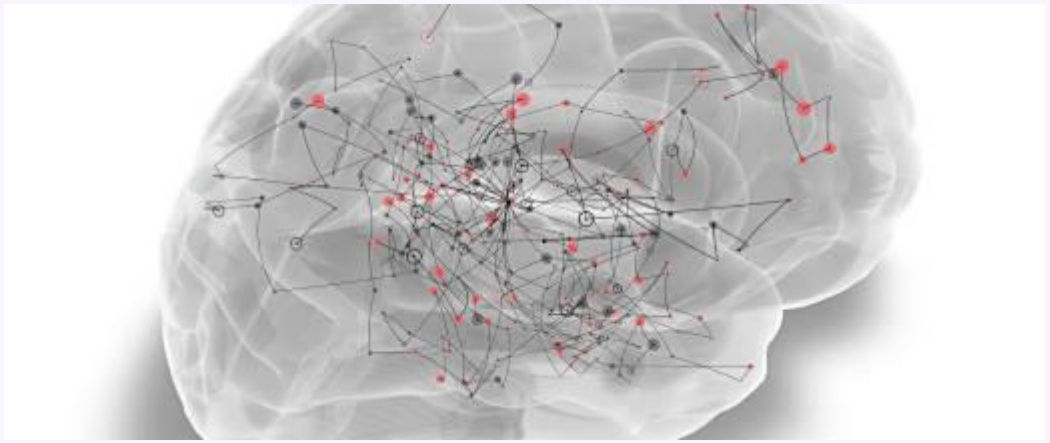
Challenge and Optimization

Load balancing issue



Expert hunger

Gated networks may tend to favor a small number of "strong experts", resulting in low utilization of other experts, a phenomenon known as "expert hunger".



Load balancing

To optimize the utilization of experts, load balancing regularization terms are introduced, such as the auxiliary loss of Switch Transformer, to balance the load of experts.

Training Stability Challenge

High communication costs

When experts are distributed across different devices, communication costs are high; Dynamic activation leads to complex gradient calculation, which needs to be optimized with a distributed training framework.

Gradient calculation complexity

Dynamic activation of experts increases the complexity of gradient calculation, requiring the design of efficient gradient aggregation algorithms, such as gradient thinning, to reduce communication burden.

Optimization of reasoning efficiency

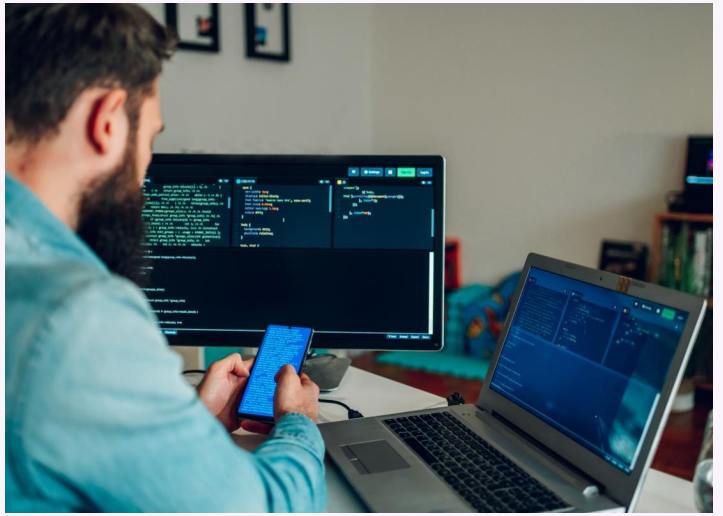
Inference logic complexity

Dynamic selection of experts increases the complexity of inference logic. To optimize inference efficiency, precompilation can be performed for expert routing.



Pre compiled expert routing

By pre compiling expert routing, the amount of routing computation during inference is reduced, achieving efficient inference. Meanwhile, this helps to expose potential issues in advance.



09

Summary of MOE

Summary of Technical Advantages

- Capacity efficiency win-win: MOE technology achieves low computational complexity with a large number of parameters by dynamically selecting experts, effectively balancing the capacity and efficiency of the model and providing strong support for large-scale language models.

- Flexible and efficient scaling: With an increase in the number of experts, MOE can easily handle more complex tasks while maintaining high computational efficiency, providing greater flexibility and possibilities for model scaling.

- Automatic load balancing: Through the selection mechanism of the gate controlled network, MOE automatically achieves load balancing among experts, effectively avoiding the problem of some experts being overloaded while others are idle, and improving the overall stability and performance of the model.

- Adapting to multiple tasks: Whether it is NLP or multimodal tasks, MOE has demonstrated excellent adaptability. Being able to dynamically adjust expert configuration according to different task requirements, ensuring the efficiency and accuracy of the model.

Application prospects and prospects

- Large Language Model: MOE, with its efficient ability to expand the number of parameters, assists in the construction of large language models, meets the market's demand for advanced language processing capabilities, and promotes further development of NLP technology.
- Multimodal model: By integrating multimodal processing capabilities, MOE promotes the fusion of visual, speech, and other types of data, providing comprehensive perception and cognitive abilities for smart devices and enriching user experience.
- Edge computing: In a resource constrained environment, MOE dynamically adjusts the number of experts or parameters to achieve flexible control of the amount of computing and provide efficient model support for edge computing.
- Customized services: By utilizing the gate control mechanism of MOE, enterprises can customize model services according to user needs, provide personalized solutions, and explore new business models and market opportunities.

Future optimization direction

- Lightweight design: In the future, MOE will develop towards lightweight direction by optimizing the gate network and expert structure, reducing parameters and computational complexity, and improving the overall efficiency of the model.
- Dynamic routing optimization: researching more efficient dynamic routing algorithms, reducing inference complexity, improving inference speed, while ensuring model performance and accuracy.
- Distributed training and deployment: With the expansion of model scale, distributed training and deployment have become necessary. In the future, we will optimize the distributed training algorithm and deployment strategy of MOE.
- Enhanced interpretability: Future research will focus on the interpretability of MOE, improving the design of gating mechanisms and expert structures to enhance the transparency and comprehensibility of the model decision-making process.